RKNN3-Toolkit API Reference

ID: RK-YH-YF-412

Release Version: V1.2.0

Release Date: 2026-10-23

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This is a RKNN3-Toolkit API Reference.

RKNN3-Toolkit is a development kit that provides users with model conversion, inference and performance evaluation on PC platforms.

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description	Reviewer
V0.2.0	HPC Team	2025-8-6	Initial version	Vincent
V0.3.0b0	NN Team	2025-9-12	Update the interface descriptions for load_rknn/init_runtime/inference/list_devices	Vincent
V0.4.0b0	NN Team	2025-10-18	Update rknn.config new param "rknn_build_parallelism", "llm_head_dtype" and "llm_head_quantized_method" description	Vincent
V0.5.0	NN Team	2025-12-01	- Support for accuracy analysis on connected target boards. - Updated init_runtime/inference API. - Add kvcache_controller/query API.	Vincent
V1.0.0	NN Team	2026-01-10	Release version.	Vincent
V1.0.1	NN Team	2026-02-13	Added LoRA support.	Vincent
V1.0.2	NN Team	2026-03-13	Added RK3572 platform description.	Vincent
V1.0.4	NN Team	2026-05-11	Update accuracy analysis new param "analysis_core_mask" and the `llm_config` configuration item.	Vincent
V1.1.0	NN Team	2026-06-29	- Removed Caffe/TensorFlow/TensorFlow Lite/DarkNet/PyTorch framework support, keeping only ONNX; - The standalone accuracy_analysis API is no longer maintained; use inference(accuracy_analysis=True) instead. - Updated system dependency description. - Added reg_custom_op API.	Vincent
V1.2.0	NN Team	2026-10-23	Updated API descriptions for paged KV Cache, LoRA, MoE, and ONNX→PyTorch model generation and rebuild.	Vincent

Contents

RKNN3-Toolkit API Reference

1 Requirement

1.1 Applicable chip model

1.2 Requirements Dependencies

1.3 Supported Deep Learning Framework

2 RKNN3-Toolkit Workflow Description

2.1 Main Workflow: Model Conversion, Export, and Optional Debugging

2.2 Rebuild Workflow: Rebuilding Models Based on Build Cache
2.3 load_ctx Workflow: Loading Context and Debugging an Exported Model
2.4 RKNN Object and Resource Lifecycle
3 Model Configuration API
3.1 Experimental Parameters
3.2 Deprecated Parameters
3.3 llm_config
4 Model Loading APIs
4.1 Loading ONNX model
4.2 Loading LLM model
5 Model Building and Quantization APIs
5.1 Building RKNN model
5.2 Rebuild RKNN model
6 Export RKNN model
7 Runtime Initialization and Inference APIs
7.1 Loading RKNN model
7.2 Initialize the runtime environment
7.2.1 Allocate and grow paged KV Cache
7.3 Register Custom Operator
7.4 Inference with RKNN model and accuracy analysis
8 Accuracy, Performance, and Memory Evaluation APIs
8.1 Accuracy Analysis
8.2 Evaluate model performance
8.3 Evaluating memory usage
8.4 List Devices
9 LLM and VLM Conversion APIs
9.1 LoRA Loading, Enabling, and Disabling
9.1.1 Load LoRA
9.1.2 Enable LoRA
9.1.3 Disable LoRA
9.2 Query Function
9.2.1 Query Command Types
9.3 KVCache Controller
9.3.1 Obtain KVCache Control Inputs
9.3.2 Clear KVCache Status
9.3.3 Roll back cache position

9.3.4 Bind and Automatically Complete Control Inputs

10 ONNX-to-PyTorch Model Generation and Rebuild

10.1 Generate a PyTorch Model

10.2 Use the PyTorch Simulator

10.3 Load and Edit a Generated Model

10.4 Rebuild RKNN from Generated Artifacts

10.5 Quantization Parameter Storage and Replacement

10.6 Example and Notes

11 RKQuantizer

11.1 API Description

11.2 Calibration Dataset Preparation

11.3 AWQ Configuration

11.4 Mixed-Precision Quantization

11.5 GRQ Quantization Memory Usage

1 Requirement

1.1 Applicable chip model

RKNN3-Toolkit currently supports model conversion for two categories of chips: **SoC chips with an integrated NPU** and **coprocessor chips**.

- **SoC chips with an integrated NPU**: currently supports RK3572.
- **Coprocessor chips**: currently supports RK1820, RK1828.

Different chip types differ in their use with RKNN3-Toolkit as follows:

Function	Related API or parameter	SoC chip with an integrated NPU	Coprocessor chip
On-board features such as on-device inference, accuracy analysis, memory usage analysis, and per-layer latency analysis	**rknn.init_runtime**	Supported. When the device is connected, RKNN3-Toolkit communicates with the development board for debugging through the adb interface.	Supported. When the device is connected, the coprocessor connects to the specified host chip through PCIE/USB/Ethernet, RKNN3-Toolkit connects to the host chip development board through USB adb or Ethernet abd(adb connect), and communicates with the coprocessor for debugging through the server program on the host chip development board.
LLM models, storing the rotary position embeddings on the host side	sub-parameter **`position_embeddings_host_storage`** of the `attention_config` sub-item of `llm_config` in `rknn.config`	Not supported	Supported. Storing the rotary position embeddings on the host side reduces the memory usage of the coprocessor chip at the cost of the transfer overhead of each inference.
LLM models, moving the lm_head matrix multiplication to the specified host chip for processing	sub-parameter **`device`** of the `llm_head` sub-item of `llm_config` in `rknn.config`	Not supported	Supported. The memory usage of llm_head is usually vocab_size x hidden_size x quantization coefficient. Moving it to the host chip for processing can reduce the memory usage of the coprocessor chip, but may increase the inference latency.

Note: The **specified host chip** in the table currently refers specifically to RK3588/RK3576.

1.2 Requirements Dependencies

It is recommended to meet the following requirements in the operating system environment:

Operating system version	Ubuntu22.04 (x64)	Ubuntu24.04 (x64)	macOS 13+ (Apple Silicon)
Python version	3.10	3.12	3.10 / 3.12

Note:

1. For more detail about python library dependencies, see requirements*.txt. Key dependencies include:
   - Python >= 3.10
   - numpy >= 2.0
   - onnx >= 1.16
   - onnxruntime >= 1.22.0
   - torch >= 2.2
   - protobuf >= 4.21.6, <= 6.33.6
   - psutil >= 5.9.0
   - scipy >= 1.12.0
   - opencv-python >= 4.8
   - pillow, sentencepiece, safetensors, datasets, tqdm, tabulate, pandas, prettytable, ruamel.yaml, fast-histogram
2. This document mainly uses Ubuntu 22.04 / Python3.10 as an example.
3. macOS Apple Silicon supports offline ONNX and existing LLM conversion, build, and export only. Board runtime, device discovery, NTP/ADB transport, performance evaluation, and memory evaluation are not supported. Intel Mac and Universal2 wheels are not provided. Use `requirements_cp310.txt` or `requirements_cp312.txt`; macOS uses the same dependency ranges as the corresponding Linux Python version, including SciPy and onnxruntime >= 1.22.0.

1.3 Supported Deep Learning Framework

RKNN3-Toolkit currently only supports the ONNX model. Caffe, TensorFlow, TensorFlow Lite, DarkNet, and PyTorch model support have been removed since V1.1.0.

Note:

1. For the relationship between ONNX release versions and opset versions and IR versions, please refer to the onnxruntime official website: https://github.com/microsoft/onnxruntime/blob/v1.6.0/docs/Versioning.md

2 RKNN3-Toolkit Workflow Description

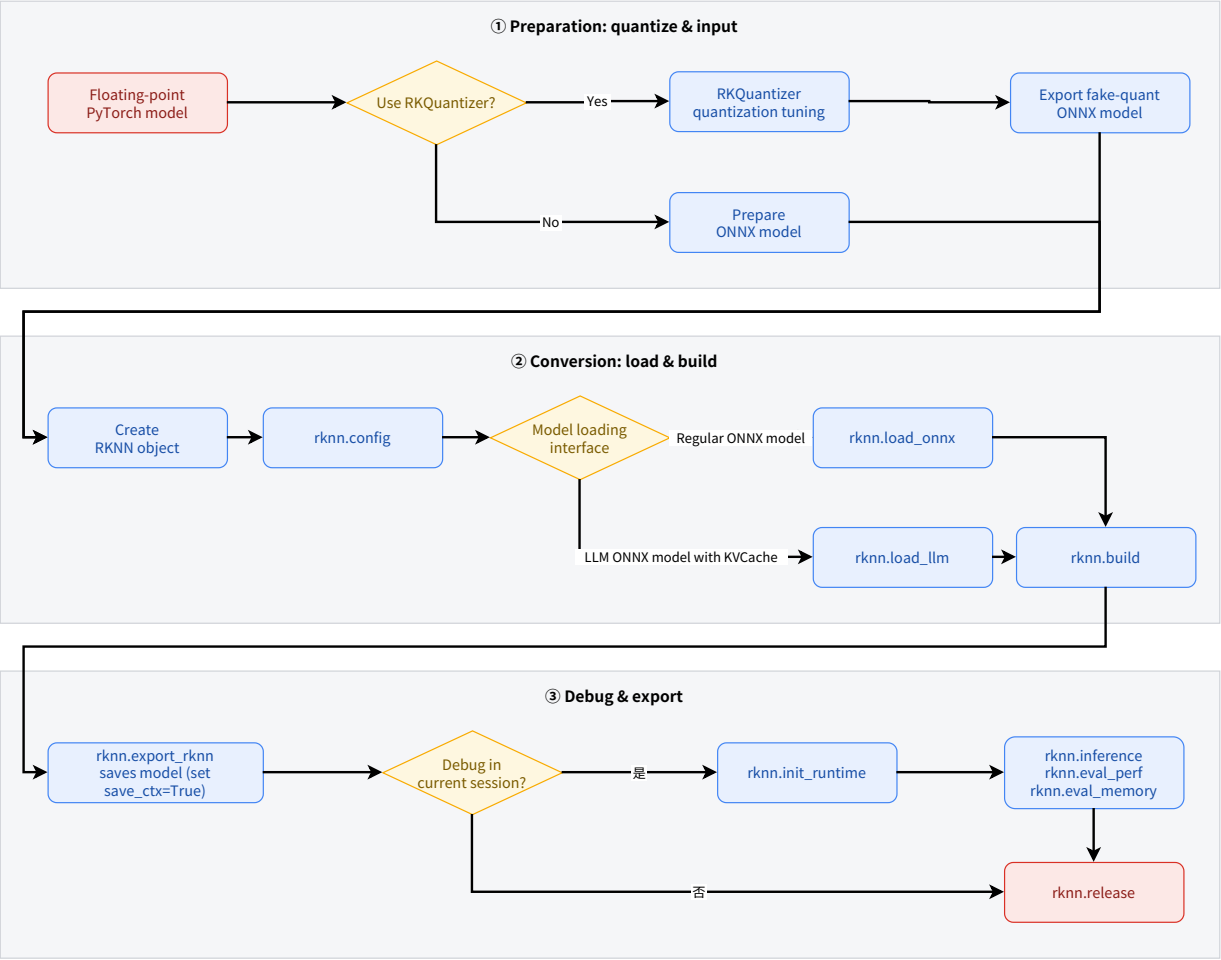
This chapter describes the common calling sequence of RKNN3-Toolkit. Model conversion, rebuild, and context-based loading debugging are three interrelated but functionally distinct workflows: use the main workflow for the first conversion; use the rebuild workflow when only adjusting part of the configuration of an already-built LLM model; use the load_ctx workflow when you need to reload an exported model and debug it.

2.1 Main Workflow: Model Conversion, Export, and Optional Debugging

When converting a model for the first time, create an `RKNN` object and call `config` to configure conversion parameters such as the target platform, quantization, and LLM options; then load the model by calling `load_onnx` or `load_llm` depending on the model type, call `build` to build it, and use `export_rknn` to export the RKNN model and its weight file. To generate an editable PyTorch model, set `generate_torch_model=True` in `config`; `export_rknn` then also creates the ONNX-to-PyTorch artifacts in the build cache.

For scenarios where you need to further debug in the Toolkit, perform accuracy analysis, or reload and debug in a later session, set `save_ctx=True` when exporting to save the context information. After building, you can also call `init_runtime` on the current `RKNN` object to initialize the runtime environment, and then call `inference`, `eval_perf`, or `eval_memory` for inference, performance, or memory debugging.

Before exporting ONNX, you may optionally use `RKQuantizer` to quantize and fine-tune the Linear weights of the PyTorch model; this step outputs a fake-quant model, which you need to export to ONNX yourself before proceeding to the RKNN conversion flow. After completing export or debugging in any workflow, call `rknn.release` to release the resources occupied by the `RKNN` object.

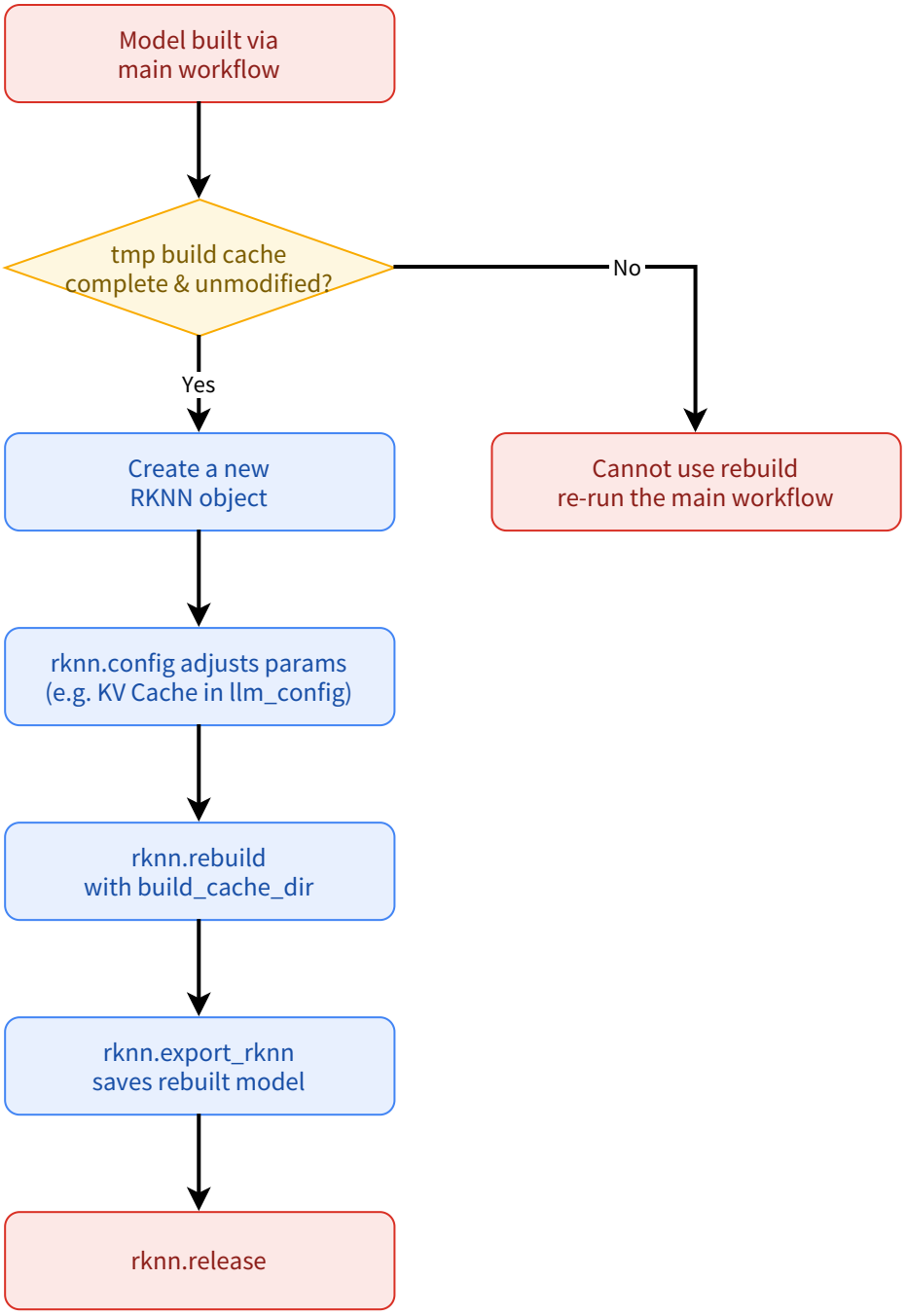


> **Note:** If you need to reload the model with `load_rknn(load_ctx=True)` for debugging later, set `save_ctx=True` in the `export_rknn` call of this workflow.

2.2 Rebuild Workflow: Rebuilding Models Based on Build Cache

`rebuild` is used to rebuild a model based on the `tmp` build cache generated in the `build` phase after the main workflow has been completed. The prerequisite for this workflow is that the original main workflow was executed successfully, and the corresponding `tmp` directory is complete and has not been modified or deleted.

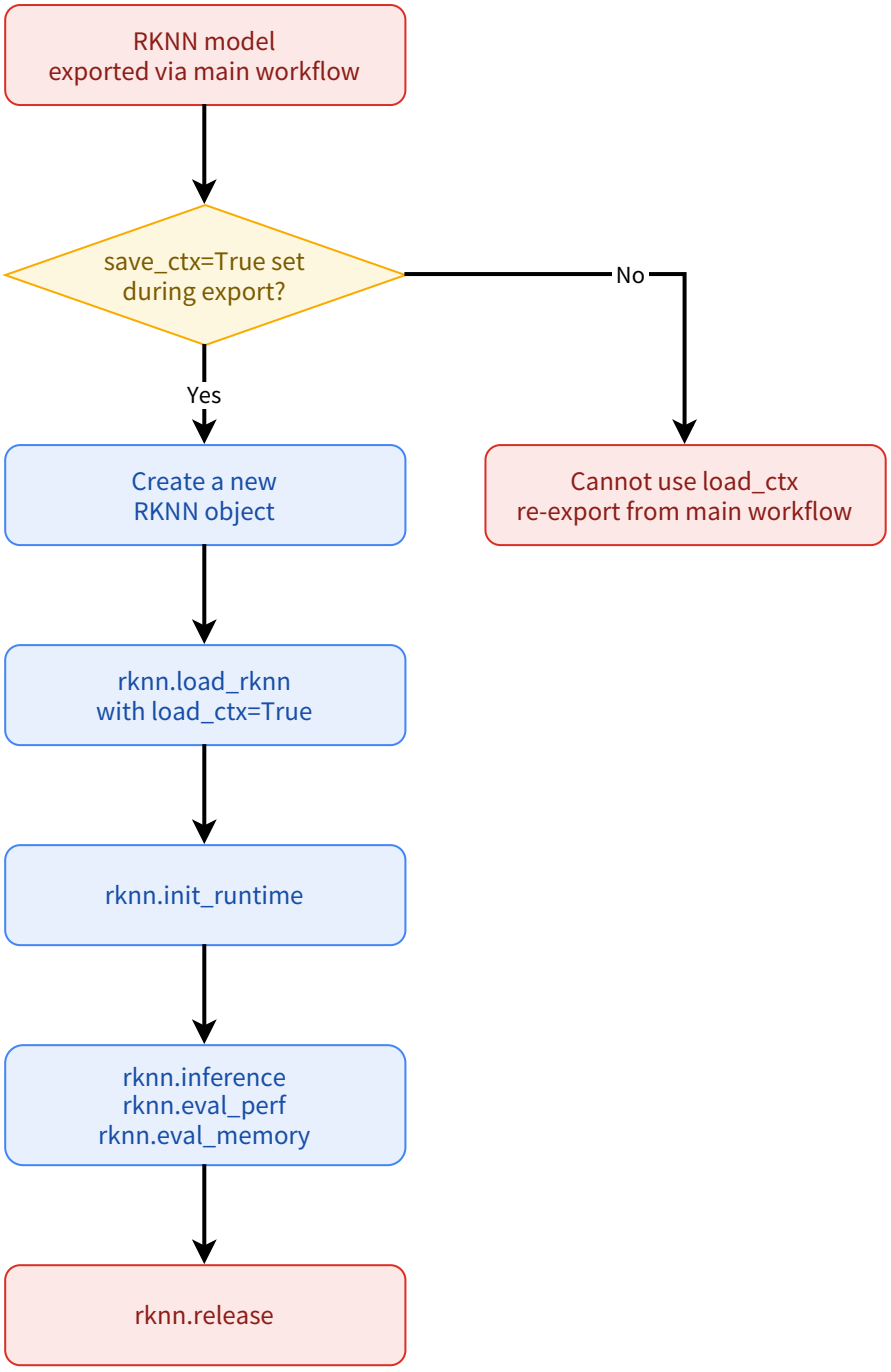
After creating a new `RKNN` object, you can use `config` to adjust the parameters supported by rebuild, such as `profile_mode` and the KV Cache-related configurations in `llm_config`; then call `rebuild(build_cache_dir=...)` and specify the original build cache directory, and finally use `export_rknn` to export the rebuilt model. A cache containing ONNX-to-PyTorch artifacts can additionally consume an edited generated model or use `quant_json` to replace compatible quantization parameters.



2.3 load_ctx Workflow: Loading Context and Debugging an Exported Model

The `load_ctx` workflow is used to reload an exported RKNN model in a new Toolkit session and perform inference, performance, or memory debugging. Its prerequisite is that the model was exported via the main workflow and `save_ctx=True` was set in `export_rknn` at that time, so that the corresponding context information file was generated.

After creating an `RKNN` object, call `load_rknn` and set `load_ctx=True` to load the model, weights, and context information; then call `init_runtime` to initialize the runtime environment, after which you can use `inference`, `eval_perf`, or `eval_memory` for debugging. Without loading the context, the model can still be used for on-device inference, but cannot be used for simulator or accuracy analysis and other context-dependent features.



2.4 RKNN Object and Resource Lifecycle

Before using the RKNN3-Toolkit APIs, you must first call `RKNN()` to create an RKNN object; after completing model conversion, debugging, or evaluation, call `release()` to release the resources occupied by the object. During initialization, you can control detailed log output through `verbose` and write logs to a specified file through `verbose_file`.

Example:

```
# Print detailed log information
rknn = RKNN(verbose=True, verbose_file=None)

...

rknn.release()
```

3 Model Configuration API

Before the RKNN model is built, the model needs to be configured first through the **config** interface.

API	**config**
Description	Set model convert parameters.
Parameter	**mean_values:** The mean values of the input. The parameter format is a list. The list contains one or more mean sublists. The multi-input model corresponds to multiple sublists. The length of each sublist is consistent with the number of channels of the input. For example, if the parameter is [[128,128,128]], it means an input subtract 128 from the values of the three channels. The default value is None, means all means is zero.
	std_values: The normalized value of the input. The parameter format is a list. The list contains one or more normalized value sublists. The multi-input model corresponds to multiple sublists. The length of each sublist is consistent with the number of channels of the input. For example, if the parameter is [[128,128,128]], it means the value of the three channels of an input minus the average value and then divide by 128. The default value is None, means all stds is one.
	quant_img_RGB2BGR: Indicates whether the RGB2BGR operation needs to be done first when loading the quantized image. If there are multiple inputs, the corresponding parameters for each input are enclosed in a list, such as [True, True, False]. The default value is False. This configuration is only valid for quantized images in jpg/jpeg/png/bmp format. It is ignored when reading npy files. Therefore, when the model input is BGR, npy data must also be in BGR format. **This configuration is only used to read quantization images during the build stage or during accuracy analysis (set accuracy_analysis=True in inference), and is not stored in the final RKNN model. Therefore, if the model input is BGR, ensure that input image data is also in BGR format before calling RKNN3-Toolkit inference or the C-API run function.**
	quantized_dtype: Quantization type. Currently supported quantization types: w4a16, w4abf16, w6a16, w8a16, w16a16, w8a8. Here "w" stands for weight quantization and "a" for activation quantization. Default is w16a16. **- w16a16:** Both weights and activations use float16 precision, equivalent to no quantization. **- w4a16:** 4-bit asymmetric weight quantization, 16-bit floating-point activation precision. **- w4abf16:** 4-bit asymmetric weight quantization, bfloat16 activation precision. **- w6a16:** 6-bit asymmetric weight quantization, 16-bit floating-point activation precision. **- w8a16:** 8-bit asymmetric weight quantization, 16-bit floating-point activation precision. **- w8a8:** 8-bit asymmetric weight quantization, 8-bit asymmetric activation quantization.

API	config
	quantized_algorithm: The quantization algorithm used when calculating the quantization parameters of each layer. Currently support: **normal**, **mmse**, **kl_divergence**. The default value is **normal**. The characteristic of **normal** quantization algorithm is fast. The recommended quantization data is generally about 20-100 pieces. with more data, the accuracy may not be further improved. The **mmse** quantization algorithm is slower due to the violent iteration method, but usually has higher accuracy than normal. The recommended quantization data is generally about 20-50 pieces. Users can also increase or decrease the amount of data appropriately according to the length of the quantization time. The **kl_divergence** quantization algorithm will take more time than normal, but will be much less than mmse. In some scenarios(when the feature distribution is uneven), better improvement effects can be obtained by “kl_divergence”. the recommended quantization data is generally about 20-100 pieces.
	quantized_method: Currently support layer, channel or group{SIZE}. Default is None. When quantized_dtype is not w16a16, a valid parameter must be set. - layer: each weight has only one set of quantization parameters. - channel: each channel of weight has its own set of quantization parameters. usually the channel will be more accurate than the layer. - group{SIZE}: On the basis of 'channel', the weight of each output channel is subdivided into multiple groups according to {SIZE} on the input channel. each group has a set of quantization parameters. Usually, group{SIZE} will be more accurate than channel. {SIZE} is the multiple value of 32 between 32 and 256, such as 'group32' or 'group128'. group{SIZE} is currently only valid when quantized_dtype = w4a16, w4abf16, w6a16, or w8a16. Note: Group quantization currently only applies to MatMul/Linear type ops and does not support convolution.
	optimization_level: Model optimization level. The default value is 3. By modifying the model optimization level, you can turn off some or all of the optimization rules used in the model conversion process. The default value of this parameter is 3, and all optimization options are turned on. When the value is 2 or 1, turn off some optimization options that may affect the accuracy of some models. Turn off all optimization options when the value is 0.
	target_platform: Specify which target chip platform the RKNN model is based on. Currently supports 'rk1820', 'rk1828', and 'rk3572'. This parameter is case-insensitive. The default value is None.

API	config
	dynamic_input: Simulate the function of dynamic input according to multiple sets of input shapes specified by the user. the format is [[input0_shapeA, input1_shapeA, ...], [input0_shapeB, input1_shapeB, ...], ...]. The default value is None. For example, the input shape of the original model is [1,3,224,224] or [1,3,height,width] or [1,3,-1,-1], but the model for deploy needs to support 3 input shapes: [1,3,224,224], [1,3,192,192] and [1,3,160,160], you can set dynamic_input=[[[1,3,224,224]], [[1,3,192,192]], [[1,3,160,160]]]. When converting to the RKNN model for inference, the input data corresponding to the shape needs to be passed in. Note: **1. This function can only be enabled when the original model itself supports dynamic input, otherwise an error will be reported.** **2. If the original model input shape itself is dynamic, only the dynamic axes can set different values.**
	core_num: The number of NPU cores required by the model. For the RK1820, the NPU core number ranges from 1 to 8. Default is 0 (auto mode). Note: 1. Transformer-based models can be configured to use the desired number of NPU cores, while other models (such as CNN models) are recommended to use 1 core. 2. In auto mode, models loaded via **load_llm** will use 8 NPU cores, while all other cases will use 1 NPU core. 3. The RK3572 platform features a single-core NPU and only supports setting this parameter to 1.
	rknn_build_parallelism: The number of parallel threads for compiling the rknn model. The default value is -1, which indicates automatic configuration. No more than 32 threads will be used during automatic configuration.
	input_attrs: Model deployment input properties. Default value is {}. The format is: {input_name: {attr_name: attr_value, ...}, ...}, e.g.: {'input_name': {'dtype': 'float32', 'layout': 'NCHW', 'shape': [1, 3, 224, 224]}, ...} attr_name: dtype, layout, shape, stride, etc.
	output_attrs: Model deployment output properties. Default value is {}. The format is: {output_name: {attr_name: attr_value, ...}, ...}, e.g.: {'output_name': {'dtype': 'float32', 'layout': 'NCHW', 'shape': [1, 1000]}, ...} attr_name: dtype, layout, shape, stride, etc.
	generate_torch_model: Whether `export_rknn` generates an editable PyTorch model, a separate weight store, and a regenerated ONNX model from the current build cache. Default is False. Artifacts are written to `<build_cache_dir>/torch_model` and can be used by the PyTorch Simulator, for model debugging, and by `rebuild`.
	llm_config: Configure LLM-related optimization options. The configuration format is a dictionary. See section 3.3 for detailed configuration instructions. After configuration, you need to call the `rknn.load_llm` interface to load the model for the changes to take effect.
Return Value	None.

The sample code is as follows:

```
# model config
rknn.config(mean_values=[[103.94, 116.78, 123.68]],
            std_values=[[58.82, 58.82, 58.82]],
            target_platform='rk1820')
```

3.1 Experimental Parameters

The following parameters are experimental features and may be adjusted or removed in future versions:

Parameter	Description
custom_string	Allows users to configure a string and save it in the RKNN model, which can be queried and retrieved via subsequent CAPI interfaces. Currently not supported. Default is None.
cvt_conv_streaming	Provide a list of output tensor names of convolution OPs to convert the specified convolutions into streaming convolutions. The streaming convolution cache will be managed internally by the NPU, offering better inference performance than external cache management. Supports 1D/2D convolutions. Upon successful conversion, the model will additionally generate a `Conv_streaming_Tc` input to control the effective length of the current inference. Default is an empty list. - Currently, streaming inference is only supported in a single dimension. For 2D convolutions, the conversion will automatically detect whether the streaming axis is on Height or Width. If both axes meet the streaming requirements, Width is used as the streaming axis by default. For 2D convolutions, it is recommended to design the model with Width as the streaming axis for better inference performance. The streaming axis requires `(kernel-1)*dilation <= left_pad` and the right pad must be 0. Manual specification of the streaming axis is not currently supported. - If the model contains multiple groups of streaming convolutions with different specifications (e.g., when the model has an upsample module, the scales of preceding and following streaming convolutions may differ), multiple groups of convolution OP output tensor names can be provided, generating multiple `Conv_streaming_Tc_{N}` inputs. - For ONNX models with explicit streaming convolution cache inputs/outputs, users need to specify valid outputs and remove streaming convolution outputs when using the `load_onnx` interface, and configure the `input_initial_value` parameter in config to set the corresponding streaming convolution cache inputs to all zeros. For users of the `load_llm` interface, they currently need to manually remove the streaming convolution cache outputs from `onnx.model.graph.output` externally. Refer to the Qwen3.5 model example for this usage. - Single-group example: `cvt_conv_streaming=['Conv0_output0', 'Conv1_output0']` - Multi-group example: `cvt_conv_streaming=[['Group0_conv0_output0', 'Group0_conv1_output0'], ['Group1_conv0_output0', ...], ...]`

Parameter	Description
disable_logN_rcclReduce	When set to True, disables the multi-core parallel rcclReduce communication operation, affecting the latency of rcclReduce communication operators. Default is False.
distribute_full_search_space	Model splitting search space. When set to True, the multi-core splitting step will try different op retrieval orders to find the optimal performance solution. Default is False.
distribute_strategy	Model splitting strategy. Currently supports 'less_subgraph', 'best_perf', 'less_mem'. When splitting a multi-core execution graph, 'less_subgraph' minimizes the number of subgraphs; 'less_mem' minimizes constant memory usage; 'best_perf' splits with performance priority, allowing weights to be copied across multiple cores (e.g., allowing convolution splitting along HW dimensions). Default is 'less_mem'.
float_dtype	Configures the float type for RKNN. Currently supports 'float16', 'bfloat16'. Default is 'float16'.
inputs_yuv_fmt	Adds a YUV->RGB conversion operator for specified inputs, supporting 'nv12' and 'nv21' YUV input types. Configured as a list of the same length as the number of inputs; set to None for inputs that do not need conversion. For example, if a model has two inputs and the second requires 'nv12' conversion, configure as `[None, 'nv12']`. Default is None. This feature is not currently supported.
input_initial_value	Configures initialization constants for model inputs, reducing input transfer overhead during model inference. Supports dynamic shape scenarios where different initialization constants can be assigned to different shapes. The format is a 2-level nested list, where the second-level list must have the same number of elements as the ONNX model inputs. Allowed types are None and np.ndarray; None indicates that the corresponding input does not need an initial constant, and np.ndarray must use the same data type as the ONNX input. - When used with dynamic shapes, ensure the order of multiple parameter groups matches the order of dynamic shape groups. Default is None. - Example without dynamic shapes: `input_initial_value=[[None, np.arange(16, dtype=np.int32)]]`. - Example with dynamic shapes: `dynamic_input=[[[1,3,192,192], [1,16]], [[1,3,384,384], [1,64]]], input_initial_value= [[None, np.arange(16, dtype=np.int32)], [None, np.arange(64, dtype=np.int32)]]`

Parameter	Description
merge_mean_std_to_conv	Incorporates the input `mean` and `std` parameters into the first convolution, eliminating the need for additional normalization during inference and improving the model's overall inference performance. If the first convolution has `pads` parameters, this operation only fuses `std` (not `mean`) to avoid numerical deviations at HW dimension edges. When enabled, it applies to all inputs. Default is False.
model_pruning	Performs lossless pruning on the model. The effective OP is convolution. When enabled, it attempts to remove all-zero input channels or all-zero output channels, reducing the size and computation of the converted RKNN model. Typically requires training or post-training to construct channel sparsity in convolutions; unprocessed models usually lack this property. Default is False.
op_quantized_dtype	Specifies the quantization dtype for specific ops. Default is an empty dictionary. - Currently only supports specifying quantization dtype for Gemm/Matmul/Linear type ops - Supported quantization dtypes: 'w4a16', 'w6a16', 'w8a16', 'w16a16' - Input format is a dictionary; the key can be a node name, node output name, or constant name. For example: {'Conv': 'w8a16'}, {'Conv_output0': 'w8a16'}, {'onnx::weight123': 'w8a16'}
profile_mode	When configured, the exported RKNN model retains additional intermediate information. This must be enabled for accuracy analysis functionality. Default is False.
sdpa_qk_scale	Applies a scale factor to Q and K in the Attention op to avoid potential overflow during exSDPA computation. The format is a dictionary where the key is the exSDPA op name and the value is a list of scaling coefficients, e.g., `{'exSDPA_op_name' : [float(1/50), float(1/50)]}`. Default is an empty dictionary, meaning no additional scaling is applied.
shard_map	Specifies the multi-core splitting method for specific ops. Currently not supported. Default is an empty dictionary.

Parameter	Description
subgraph_dtype_target	Configures the inference precision type of a specified segment in the model to the target type. This parameter currently only takes effect when quantization is enabled and currently only supports specifying that the corresponding segment uses w16a16 (equivalent to Float16) for inference. The parameter is a two-level list, e.g., `[[subgraph0_inputs, subgraph0_outputs, dtype], [subgraph1_inputs, subgraph1_outputs, dtype], ...]`, where `subgraph_inputs` is a list containing one or more input tensor names of the segment, `subgraph_outputs` is a list containing one or more output tensor names of the segment. Tensor names can be obtained from the table generated by the accuracy analysis function. `dtype` indicates the inference precision type. Note 1: In particular, when `subgraph_inputs` and `subgraph_outputs` are the inputs and outputs of a specific op, this is equivalent to specifying that op to use the assigned inference precision type. Note 2: It is not mandatory for `subgraph_inputs` and `subgraph_outputs` to include all inputs and outputs of the corresponding segment. In this case, some ops in the segment will use the corresponding type. The judgment logic is: if an op's input has an inference association with `subgraph_inputs` and the op's output has an inference association with `subgraph_outputs`, the op will be configured to the specified inference precision type.
sparse_infer	When the convolution weights in the model meet sparsity requirements, the NPU enables sparse inference. Currently not supported. Default is False.
simulate_lut	When enabled, the simulator uses a lookup table to simulate NPU behavior for activation function operators during inference, improving accuracy consistency with the NPU but reducing simulator inference performance. Default is False.

3.2 Deprecated Parameters

The following parameters are deprecated:

Parameter	Description
attn_nodes_config	Feature deprecated.
auto_hybrid_cos_thresh	Feature deprecated.
auto_hybrid_euc_thresh	Feature deprecated.
kvcache_dtype	Please use `kvcache_dtype` in the `attention_config` sub-item of the `llm_config` parameter instead.
kvcache_store_method	Automatically determined internally based on `kvcache_dtype`; no longer requires configuration.
kvcache_group_size	Automatically determined internally based on `kvcache_dtype`; no longer requires configuration.

Parameter	Description
kvcache_residual_depth	Automatically determined internally based on `kvcache_dtype`; no longer requires configuration.
max_ctx_len	Please use `kvcache_buffer_len` in the `attention_config` sub-item of the `llm_config` parameter instead.
max_position_embeddings	Please use `max_position_embeddings` in the `attention_config` sub-item of the `llm_config` parameter instead.
position_embeddings_host_storage	Please use `position_embeddings_host_storage` in the `attention_config` sub-item of the `llm_config` parameter instead.
remove_reshape	Feature deprecated.
single_core_mode	Feature deprecated.

3.3 llm_config

For LLM models, `rknn.config` has added the `llm_config` parameter. The LLM config has four dictionary sub-items: `attention_config`, `llm_head`, `vocab`, and `keep_one_logit`.

---

attention_config: This is used to configure the "**KV cache**" definition and the inference method of the Attention op. It accepts lists, and multiple dictionaries can be included in the list to specify the specific parameters of various Attention structures. The sub-dictionaries of attention_config are defined as follows:

Parameter	Function	Dtype:default value	Notice
mask_name	Specify the mask name. In subsequent processes, the Attention op will determine its configuration by recognizing the mask input. **This parameter is mandatory!**	String: 'attention_mask'	Different types of Attention structures are not allowed to share the same mask input.

Parameter	Function	Dtype:default value	Notice
kvcache_buffer_len	Specifies the KV Cache length. During inference, if the context length exceeds the set length, it will be overwritten cyclically according to the first-in-first-out principle. A larger configuration value will result in more memory usage for the KV Cache, and theoretically, it will perform better on long text tasks.	Int:1024	Values greater than 512 must be aligned to 512; values less than or equal to 512 must be aligned to `kvcache_group_size`.
tokens_per_page	Number of tokens per paged KV Cache page; set to `None` to use the original contiguous KV Cache.	Int:512	A nonpositive value or SlidingAttention uses `kvcache_buffer_len`. The resolved value must be a multiple of `max(MacC, MacK)`.
request_num	Number of paged Attention requests.	Int:1	Must be a positive integer. Multiple requests currently support decode only, with `Tc` of 0 or 1 per request.
kvcache_dtype	Specifying the data type for context storage and computation affects inference performance and cache memory usage. Regarding memory usage, Float16 > Int8_to_F16 > Int4_to_F16	String: 'Float16'	Support Float16, Int8_to_F16, Int4_to_F16
attention_type	Specify Attention type	String: 'FullAttention'	Support FullAttention, SlidingAttention

Parameter	Function	Dtype:default value	Notice
sliding_window_size	Specifies the sliding window length for Sliding Attention. This parameter has no effect when the attention type is FullAttention.	Int: -1	/
position_name	Specify the input name for the position id, used to extract position-encoded feature values. For earlier models, position encoding may not exist; in this case, a default value of None is allowed.	String: 'position_ids'	Different types of Attention structures do not allow sharing the same position_ids input.
max_position_embeddings	Specifies the maximum length of the positional encoding. An error will be thrown if the context length exceeds the set length during inference.	Int: 8192	If `position_name` is not None and `kvcache_buffer_len` exceeds this value, it is automatically updated to `kvcache_buffer_len`.
position_embeddings_host_storage	Specifies the storage location of positional embeddings (RoPE). When set to `False`, the positional embeddings reside in device memory; when set to `True`, the positional embeddings are stored in host memory.	Bool: False	`True`: Store in host memory; `False`: Store in device memory

Parameter	Function	Dtype:default value	Notice
mrope_type	When the LLM model is a special mrope type, the mrope function can be configured and enabled. Currently, the ['Qwen2.5-VL', 'Qwen3-VL', 'Qwen3.5'] type is supported. **Note:** When enabling mrope, position_name must be configured, and this position_name is a one-dimensional input. After enabling the mrope function, the toolkit will automatically generate an input with [seq_len, 3] dimensions.	Strings: None	/
mrope_section	Configure the mrope section information. For Qwen2.5-VL, Qwen3-VL, and Qwen3.5 models, the corresponding configuration values can be found in the model's configuration file config.json.	list: None	/
mrope_new_id_name	Configure mrope to generate new position names. For the currently supported ['Qwen2.5-VL', 'Qwen3-VL', 'Qwen3.5'], a new input with dimension [seq_len, 3] will be generated.	Strings: None	/

llm_head: This is used to configure and enable optimization behavior for the LLM model's Head layer. (The Head layer is typically a single Linear layer with a large number of parameters). Multiple LLM Head layers are not currently supported; this will be added in future updates.

Parameter	Function	Dtype:default value	Notice
name	Configure the weight names of the LLM head layer.	String: auto	Currently, only `auto` is supported; specifying a specific value is not yet supported.
device	Configuring computing devices for the LLM head layer	String: rk1820	support rk1820, rk3588, rk3576
quantized_dtype	Configure the quantization type for the LLM head layer. The LLM head layer is quite large; using quantization can effectively reduce memory usage.	String: w6a16	support w16a16, w4a16, w6a16, w8a8. When the platform is rk3576 or rk3588, it will be forcibly changed to w8a8.
quantized_method	Configure the quantization method for the LLM head layer.	String: group32	Supports layer, channel, and group{Size}. When the platform is rk3576 or rk3588, it will be forced to change to channel.

vocab: Used to configure and enable the LLM model's optimization behavior for the dictionary layer.

Parameter	Function	Dtype:default value	Notice
index_name	Configure the vocab layer index input name	String: auto	Currently, only the `auto` option is supported; specifying a different option is not yet supported.
tie_embeddings	Configure the tie embedding parameter for the vocab layer. When tie_embeddings is True, it means that vocab and llm_head share the same array. Enabling this eliminates the need to store vocab separately, but because llm_head has quantization behavior, dequantizing it to float16 embedding will reduce inference performance.	Bool: False	/

Parameter	Function	Dtype:default value	Notice
store_on_compute_device	Configure whether the vocab is stored on the computing device. Storing the vocab on the computing device can reduce the transfer time required for each round of LLM inference, but it will significantly increase the memory usage of the computing device.	Bool: False	*Not supported in current version*

---

keep_one_logit: This is used to configure and enable single-value sampling (gather op) on a specified axis of the output. For most LLM models, only the last valid token needs to be sampled during output sampling. For example, with an input token sequence length of 100, the 100th output sequence is valid, and all other outputs are discarded. Therefore, this discarding behavior can be moved earlier in the `llm_head`, reducing the computational cost of the linear layer in the `llm_head` and the memory usage required during computation.

Parameter	Function	Dtype:default value	Notice
output_name	Specify output name	String: None	Set None to disable 'keep_one_logit' function.
idx_name	Generate the names of the gather indices input and add them to the model's input.	String: num_logits_to_keep	/
axis	Specify the axis for which gather is active	Int: None	/

The sample code is as follows:

```python
from rknn.api import RKNN, DEFAULT_RKNN_LLM_CONFIG
rknn = RKNN(verbose=True)
my_config = DEFAULT_RKNN_LLM_CONFIG.copy()
rknn.config(target_platform='rk1820', llm_config=my_config)

# default config is
DEFAULT_RKNN_LLM_CONFIG = {
'attention_config': [{
    'mask_name'              : 'attention_mask',
    'kvcache_buffer_len'     : 1024,
    'tokens_per_page'        : 512,
    'request_num'            : 1,
    'kvcache_write_policy'   : 'WRITE_THROUGH',
    'kvcache_dtype'          : 'Float16',
    'attention_type'         : 'FullAttention',
    'sliding_window_size'    : -1,
    'position_name'          : 'position_ids',
    'max_position_embeddings': 8192,
    'position_embeddings_host_storage': False,
    'mrope_type'             : None,
    'mrope_section'          : None,
    'mrope_new_id_name'      : None,
}],
'llm_head':[{
    'name'                  : 'auto',
    'device'                : 'RK1820',
    'quantized_dtype'       : 'w6a16',
    'quantized_method'      : 'group32',
}],
'vocab':[{
    'index_name'            : 'auto',
    'tie_embeddings'         : False,
    'store_on_compute_device': False,
}],
'keep_one_logit': [{
    'output_name': None,
    'idx_name'   : 'num_logits_to_keep',
    'axis'       : None,
}],
}
```

4 Model Loading APIs

RKNN3-Toolkit currently only supports loading ONNX models and the dedicated LLM model loading interface. The loading interfaces for Caffe, TensorFlow, TensorFlow Lite, DarkNet, and PyTorch have been removed since V1.1.0.

4.1 Loading ONNX model

API	**load_onnx**
Description	Load ONNX model.
Parameter	**model:** The path of ONNX model file (suffixed with ".onnx").
	inputs: The input node (operand name) of model, input with multiple nodes is supported now. All the input node string are placed in a list. The default value is None, means get from model.
	input_size_list: The shapes of input node, all the input shape are placed in a list. If inputs set, the input_size_list should be set also. default is None.
	input_initial_val: Set the initial value of the model input, the format is ndarray list. The default value is None. Mainly used to fix some input as constant. For the input that does not need to be fixed as a constant, it can be set to None, such as [None, np.array([1])].
	outputs: The output node (operand name) of model, output with multiple nodes is supported now. All the output nodes are placed in a list. The default value is None, means get from model.
Return Value	0: Import successfully.
	-1: Import failed.

The sample code is as follows:

```
# Load the arcface onnx model in the current path
ret = rknn.load_onnx(model='./arcface.onnx')
```

4.2 Loading LLM model

API	**load_llm**
Description	Load the LLM model. The LLM model is exported from the rkllm3_model_zoo project. (For example, rkllm3_model_zoo/examples/Qwen2_5/python/export_llm.py)
Parameter	**model**: LLM model path (.onnx).
	config: LLM config file path.
	seq_lens: LLM input sequence lengths. Default: [1, 128].
Return Value	0: Import successfully.

API	load_llm
	-1: Import failed.

The sample code is as follows:

```
# Load Qwen2.5-0.5B locally
ret = rknn.load_llm(model='./Qwen2.5-0.5B.onnx',
                    config='./Qwen2.5-0.5B.pkl')
```

5 Model Building and Quantization APIs

5.1 Building RKNN model

API	build
Description	Build corresponding RKNN model according to imported model.
Parameter	**do_quantization:** Whether to quantize the model. The default value is True.
	dataset: A input dataset for rectifying quantization parameters. Currently supports text file format, the user can place the path of picture(jpg or png) or npy file which is used for rectification. A file path for each line. Such as: a.jpg b.jpg or a.npy b.npy If there are multiple inputs, the corresponding files are divided by space. Such as: a.jpg a2.jpg b.jpg b2.jpg or a.npy a2.npy b.npy b2.npy Note: It is generally recommended to select the quantization image which is consistent with the prediction scene.
Return value	0: Build successfully.
	-1: Build failed.

The sample code is as follows:

```
# Build and quantize RKNN model
ret = rknn.build(do_quantization=True, dataset='./dataset.txt')
```

5.2 Rebuild RKNN model

API	rebuild
Description	Rebuild an RKNN model. Before rebuilding, you can call config to change the behavior of the exported RKNN model. Currently, the **profile_mode** parameter in config and the llm_config parameter support changes during the rebuild phase. Specifically, the **kvcache_buffer_len**, **kvcache_dtype**, **kvcache_store_method**, **kvcache_group_size**, **kvcache_residual_depth**, and **max_position_embeddings** parameters in llm_config's attention_config support changes during the rebuild phase. For a build cache created with `generate_torch_model`, `rebuild` can also consume an edited `torch_model/generated.py`, weight store, and quantization parameters; a structural graph change triggers recalibration and a full rebuild. **Note:** rebuild currently does not support subsequent on-device functionality, such as on-device inference and accuracy analysis.
Parameter	**build_cache_dir**: Specifies the tmp folder generated in the current execution directory during the rknn.build phase. This parameter is required.
	strict_check: Whether to perform strict checking. In some cases, the tmp directory may fail checks due to minor changes or interruption during the rebuild process. In such cases, set strict_check to False. Default is True.
	quant_json: Quantization manifest JSON used to replace quantization parameters. It is effective only for a build cache containing ONNX-to-PyTorch artifacts. If omitted, the original build-cache quantization parameters are used. Compatible parameters can be replaced directly when graph structure is unchanged. See Chapter 10. Default is None.
	dataset: Calibration dataset required when an edit to `generated.py` changes the structure of a quantized graph. It can be omitted when only compatible quantization parameters are replaced and graph structure is unchanged. Default is None.
Return Value	0: Build successfully.
	-1: Build failed.

The sample code is as follows:

```
# Rebuild RKNN model and export
rknn = RKNN(False)
ret = rknn.rebuild(build_cache_dir='./tmp', strict_check=True)
rknn.export_rknn('mobilenet_v1.rknn')
```

6 Export RKNN model

The RKNN model built by 'build' interface can be saved as a file, it can be used for model deployment.

API	**export_rknn**
Description	Save the RKNN model to the specified `.rknn` file and its weights to the matching `.weight` file. If `save_ctx=True`, context is also saved as `.rknn.ctx`. When `config(generate_torch_model=True)` is enabled, this API additionally generates the PyTorch artifacts described in Chapter 10; it returns -1 if artifact generation fails.
Parameter	**export_path:** The path of generated RKNN model file.
	save_ctx: Export context info (for load_rknn's load_ctx). Default: False.
Return Value	0: Export successfully.
	-1: Export failed.

The sample code is as follows:

```
# save the built RKNN model as a mobilenet_v1.rknn file and model weight as a
mobilenet_v1.weight in the current path
ret = rknn.export_rknn(export_path='./mobilenet_v1.rknn')
```

7 Runtime Initialization and Inference APIs

7.1 Loading RKNN model

API	load_rknn
Description	Load RKNN model. After loading the RKNN model, there is no need to perform the steps of model configuration, loading model and building RKNN model. If load_ctx=False, the loaded model is limited to connecting to the NPU hardware for inference or performance data acquisition. It can not be used for simulator or accuracy analysis. If load_ctx=True, the loaded model can connecting to the NPU hardware for inference or performance data acquisition. also can be used for simulator or accuracy analysis.
Parameter	**model_path:** The path of RKNN model file.
	weight_path: The path of weight file of RKNN model.
	lora_weight_path: A single `.lora_weight` file path, a directory containing multiple `.lora_weight` files, or a list of paths.
	load_ctx: Whether to load context information. Default: False.
Return Value	0: Load successfully.
	-1: Load failed.

The sample code is as follows:

```
# Load the mobilenet_v1 RKNN model in the current path
ret = rknn.load_rknn(model_path='./mobilenet_v1.rknn',
weight_path='./mobilenet_v1.weight')
```

7.2 Initialize the runtime environment

Before inference or performance evaluation, the runtime environment must be initialized. This interface determines the type of runtime (hardware platform or software simulator).

API	init_runtime
Description	Initialize the runtime environment.
Parameter	**target:** Target hardware platform. Defaults to None (model runs on the simulator). When a specific platform is specified, the model executes on the corresponding device. Supports combined mode ['', 'simulator'] to run concurrently on both physical devices and the simulator. Primarily used for multi-turn accuracy analysis of large language models, where the key-value cache (KV Cache) is retained in the simulator.
	device_id: Device identity number, if multiple devices are connected to PC, this parameter needs to be specified which can be obtained by calling "**list_devices**" interface, the device ID must be either "rk3576" or "rk3588". The default value is None.

API	init_runtime
	core_mask: The core of the model's target inference. If target is set, this parameter needs to be specified. It supports mask values from 0x1 to 0xff. The default value is -1.
	custom_op_plugins: Path to a custom op plugin shared library (.so). This plugin will be registered to the runtime context after model init. If already provided via `reg_custom_op`, there is no need to pass it again here. The default value is None. **Note:** If the model contains custom ops registered via `reg_custom_op` but the plugin path was **not** specified in `reg_custom_op`, it **must** be passed here via `custom_op_plugins`.
	simulator_backend: Software simulator backend. Supported values are `'onnx'` (default) and `'pytorch'`. The PyTorch backend runs ONNX-to-PyTorch generated artifacts and is effective only when `target=None` or `target='simulator'`.
	pytorch_artifact_dir: The `torch_model` directory containing `generated.py` and `generated_weights.pth`. Required by the PyTorch Simulator.
	pytorch_quant_json: Optional quantization manifest JSON used by the PyTorch Simulator to override quantized weights and scale/zero-point buffers in the generated model.
Return Value	0: Initialize the runtime environment successfully.
	-1: Initialize the runtime environment failed.

Note (connection method): Currently, the PC can only connect to the device via ADB mode, in one of the following two ways (see section 1.1):

- **Coprocessor chips**: The PC connects to the **specified host** (whose SoC must be RK3576 or RK3588) via ADB; the host then connects to the **coprocessor chip** (RK1820/RK1828) via PCIe or USB. In this case, set `target` to the compute chip model and `device_id` to the host device ID.
- **SoC chips with an integrated NPU**: The PC connects directly via ADB. In this case, set `target` to the compute chip model and `device_id` to that device ID.

The sample code is as follows:

```
# Initialize the runtime environment
ret = rknn.init_runtime(target=None)
```

7.2.1 Allocate and grow paged KV Cache

Use these APIs after `init_runtime(target=...)` initializes a hardware runtime for a paged Attention model. If pages are not explicitly allocated, the first inference allocates capacity for the model's full context. To reduce initial memory use, call `alloc_kvcache_pages(max_tokens)` before the first inference. The runtime must be idle while pages grow; existing page contents are retained.

API	alloc_kvcache_pages(max_tokens=None) / grow_kvcache_pages(max_tokens=None)
Description	`alloc_kvcache_pages` allocates and binds pages on the first call and only grows them on later calls. `grow_kvcache_pages` grows existing pages or performs the initial allocation. Only FullAttention page counts follow `max_tokens`; other cache types use the model's required page counts.
Parameter	**max_tokens**: Positive integer token capacity for FullAttention, no greater than the model's maximum context length. `None` selects full capacity.
Return value	0 on success; a negative runtime error code on failure. Requires a hardware runtime and a model/runtime with paged cache support.

```
rknn.init_runtime(target='rk1820')
rknn.alloc_kvcache_pages(1024)
# Grow while the runtime is idle; existing page contents are retained.
rknn.grow_kvcache_pages(2048)
```

7.3 Register Custom Operator

API	reg_custom_op
Description	Register a custom operator. Only supported for ONNX models. Should be called after `config` and before `load_onnx`, and must be called before `build`.
Parameters	**custom_op**: A custom operator class that implements `op_type`, `shape_infer`, and `compute`. The `op_type` is recommended to start with 'cst'.
	custom_op_plugins: Path to a custom op plugin shared library (.so). If specified here, there is no need to pass it again in `init_runtime`. If not specified here, it must be passed via `init_runtime(custom_op_plugins=...)` before any board-side runtime or accuracy analysis. The default value is None.
Return Value	0: Registration successful.
	-1: Registration failed.

Example of a custom operator class:

```python
import numpy as np
from rknn.api.custom_op import get_node_attr

class cstSoftmax:
    op_type = 'cstSoftmax'

    def shape_infer(self, node, in_shapes, in_dtypes):
        out_shapes = in_shapes.copy()
        out_dtypes = in_dtypes.copy()
        return out_shapes, out_dtypes

    def compute(self, node, inputs):
        x = inputs[0]
        axis = get_node_attr(node, 'axis')
        x_max = np.max(x, axis=axis, keepdims=True)
        tmp = np.exp(x - x_max)
        s = np.sum(tmp, axis=axis, keepdims=True)
        outputs = [tmp / s]
        return outputs
```

Recommended usage for `custom_op_plugins`:

```python
# Way 1: pass plugin path in reg_custom_op; init_runtime picks it up automatically
rknn.reg_custom_op(cstSoftmax, custom_op_plugins='/path/to/libcustom_op.so')
rknn.init_runtime(target='rk1820')

# Way 2: pass plugin path directly in init_runtime
rknn.reg_custom_op(cstSoftmax)                        # no plugin path here
rknn.init_runtime(target='rk1820',
                  custom_op_plugins='/path/to/libcustom_op.so')
```

7.4 Inference with RKNN model and accuracy analysis

This interface kicks off the RKNN model inference and get the result of inference.

> **Note:** The standalone `accuracy_analysis` API is no longer maintained. Use `inference(accuracy_analysis=True)` for accuracy analysis.

API	**inference**
Description	Use the model to perform inference with specified input and get the inference result.
Parameter	**inputs:** Inputs list to be inferred, The object type is ndarray.
	data_format: The layout list of input data. "nchw" or "nhwc" , only valid for 4-dims input. The default value is None, means all inputs layout is "nhwc".
	accuracy_analysis: Whether to enable per-layer accuracy analysis. The default value is False. If the target parameter is set in init_runtime, on-device accuracy analysis will be performed. When performing link precision analysis, set `profile_mode=True` in the `config`.
	analysis_core_mask: Specifies the cores used for precision analysis. Currently supports integers ranging from 0x1 to 0xff. The default value is 0, which indicates that precision analysis is performed on all cores.
	output_dir: output directory, all snapshot data will be stored here. The default value is './snapshot'. If the target is not set in init_runtime, the following content will be output under 'output_dir': - Directory simulator: Save the results of each layer on simulator when the entire quantitative model is fully run (The output has been converted to float32). - Directory golden: Save the results of each layer on simulator when the entire floating-point model is completely run down. - error_analysis.txt: Record the cosine distance (entire_error and single_error) between each layer result on the simulator and the floating-point model on the simulator during the complete calculation of the quantized model. The difference between entire_error and single_error is whether the input of each layer comes from the quantization model or the floating-point model. See the error_analysis.txt file for more details. If the target is set in init_runtime, more content will be output under 'output_dir': - Directory runtime: Save the results of each layer when the entire quantitative model is fully run on the NPU (The output has been converted to float32). - error_analysis.txt: Record the cosine distance (entire_error) between each layer result on the simulator and each layer on the NPU during the complete calculation of the quantized model additionally. See the error_analysis.txt file for more details.
Return Value	results: The result of inference, the object type is ndarray list.

The sample code is as follows (refer to *examples/onnx/yolov5* for the complete code):

```
# Perform inference for a picture with object detection. Showing object class and
XY-position.
outputs = rknn.inference(inputs=[img], data_format=['nhwc'])
...
```

The result as follows:

```
    class        score      xmin, ymin, xmax, ymax
-------------------------------------------------
    person       0.881     [ 209,  244,  286,  506]
    person       0.864     [ 478,  238,  559,  526]
    person       0.829     [ 110,  238,  230,  534]
    person       0.323     [  79,  353,  122,  517]
     bus         0.705     [  92,  131,  555,  465]
```

8 Accuracy, Performance, and Memory Evaluation APIs

8.1 Accuracy Analysis

Accuracy analysis is enabled through `inference(accuracy_analysis=True)` in section 7.4; for on-device accuracy analysis, `profile_mode=True` must be configured in `config`.

8.2 Evaluate model performance

API	**eval_perf**
Description	Evaluate model performance. The model must be running on a device connected to the PC. In `config`, set `profile_mode=True`.
Parameter	**log_level:** Level of printed performance information, default 0. 0: print summary table only; 1: print detailed per-Op table and summary table; 2: print per-Op details including time, cycles and bandwidth, plus summary table.
	dynamic_idx: Index for dynamic input, only effective for dynamic shapes. Default 0.
Return Value	0: Eval successfully.
	-1: Eval failed.

The sample code is as follows:

```
# Evaluate model performance
ret = rknn.eval_perf()
```

8.3 Evaluating memory usage

API	**eval_memory**
Description	Fetch memory usage when model is running on hardware platform. The model must be running on a device connected to the PC.
Parameter	None
Return Value	0: Eval successfully.
	-1: Eval failed.

The sample code is as follows:

```
# eval memory usage
ret = rknn.eval_memory()
```

8.4 List Devices

API	list_devices
Description	List all ADB devices connected.
Parameter	None.
Return Value	Return an adb_devices list. If there are no devices connected to the PC, it will return an empty list.

The sample code is as follows:

```
adb_devices = rknn.list_devices()
```

The devices list looks like below:

```
************************
all device(s) with adb mode:
adb_devices = 0123456789ABCDEF, 196.1.1.102:1820
************************
```

9 LLM and VLM Conversion APIs

9.1 LoRA Loading, Enabling, and Disabling

9.1.1 Load LoRA

API	**load_lora**
Description	Load LoRA weights. The LoRA weights should be loaded between model load and build.
Parameters	**lora_paths**:Nested list of LoRA weight file paths (with `.safetensors` suffix). The outer list denotes mutually exclusive LoRA groups; multiple weights in one inner list share one graph computation.
	lora_config_paths:Nested list of LoRA config file paths (with `.json` suffix), corresponding one-to-one with the inner lists of **lora_paths**.
	lora_patterns: Nested list of LoRA pattern names, corresponding one-to-one with the elements in the corresponding inner lists of **lora_paths**. The names serve as keywords to enable or disable the corresponding LoRA during inference. Only one pattern in the same LoRA group can be enabled.
	lora_quantized_dtype: LoRA weight data type. Currently supported data types: int4, int6, int8, float16, and bfloat16. The default is None; when omitted, it follows **float_dtype** for float16/bfloat16, otherwise it falls back to float16.
	lora_quantized_method: LoRA weight quantized method. Currently supports layer, channel, group{SIZE}, default is None.
	lora_node_map: Optional exact LoRA node mapping. Accepts a Python dictionary or a JSON file path. When provided, the exact base node names and A/B weight keys in the mapping are used instead of prefix/postfix matching; it cannot be used together with **lora_prefix**, **lora_postfix_a**, or **lora_postfix_b**.
	lora_prefix: Prefix for LoRA weight keys. The default is 'base_model.model.'.
	lora_postfix_a: Suffix for LoRA A weight keys. The default is '.lora_A.weight'.
	lora_postfix_b: Suffix for LoRA B weight keys. The default is '.lora_B.weight'.
	lora_distribute_strategy: LoRA weight multi-core shard strategy. Currently supports 'best_perf', 'less_mem'. default is 'best_perf'.
Return Value	0: Load successful.
	-1: Load failed.

> **Note:** When **lora_node_map** is not provided, LoRA keys are matched as `prefix + converted model node name + postfix A/B`. The model node name is converted by replacing `/` with `.`, removing trailing `MatMul`, and stripping leading and trailing `.`.

Example:

```
# Load LoRA weights from the current directory
rknn.load_lora(
    lora_paths=[
        ["./slot0_lora0.safetensors", "./slot0_lora1.safetensors"],
        ["./slot1_lora0.safetensors", "./slot1_lora1.safetensors"],
    ],
    lora_config_paths=[
        ["./slot0_lora0.json", "./slot0_lora1.json"],
        ["./slot1_lora0.json", "./slot1_lora1.json"],
    ],
    lora_patterns=[
        ["slot0_lora0", "slot0_lora1"],
        ["slot1_lora0", "slot1_lora1"],
    ],
    lora_distribute_strategy='best_perf',
)
```

Example using a custom exact node mapping:

```
lora_node_map = {
    "decoder.layers.0.self_attn.q_proj": [
        [
            ["adapter.g0.p0.lora_A.weight", "adapter.g0.p0.lora_B.weight", 0],
            ["adapter.g0.p1.lora_A.weight", "adapter.g0.p1.lora_B.weight", 1],
        ],
        [
            ["adapter.g1.p0.lora_A.weight", "adapter.g1.p0.lora_B.weight", 0],
            ["adapter.g1.p1.lora_A.weight", "adapter.g1.p1.lora_B.weight", 1],
        ],
    ],
}
rknn.load_lora(
    lora_paths=lora_paths,
    lora_config_paths=lora_config_paths,
    lora_patterns=lora_patterns,
    lora_node_map=lora_node_map,
)
```

The top-level key is the base node name in the loaded IR. The first nested list indexes the LoRA group, and each record is `[a_key, b_key, weight_index]`. The A/B keys must exist in the corresponding safetensors file, and `weight_index` is the file index within that group. The same structure can be saved as a JSON file and passed as `lora_node_map='./lora_node_map.json'`.

9.1.2 Enable LoRA

API	**enable_lora**
Description	Enable the specified LoRA.
Parameters	**enable_patterns**: List of enable lora pattern names.
	scales: List of scale factors for LoRA weights, e.g. [1.0, 0.8]. Default is None, which means scales are read from the LoRA config file.
Return Value	0: Enable successful.

API	enable_lora
	-1: Enable failed.

Example:

```
# Enable the specified LoRA
rknn.enable_lora(enable_patterns=["slot0_lora0"])
```

9.1.3 Disable LoRA

API	disable_lora
Description	Disable the specified LoRA.
Parameters	**disable_patterns**: List of disable lora pattern names.
Return Value	0: Disable successful.
	-1: Disable failed.

Example:

```
# Disable LoRA
rknn.disable_lora(disable_patterns=["slot0_lora0"])
```

9.2 Query Function

API	query
Description	Queries model-related information or status.
Parameters	**cmd**： The type of query command, provided as a string.
Return Value	results： The result of the query

9.2.1 Query Command Types

COMMAND	DESCRIPTION
QUERY_ROPE_CACHE	Queries the sin/cos cache used for the model's RoPE (Rotary Position Embedding) input.

The sample code is as follows:

```
# Query the sin/cos cache used for the model's RoPE input
rope_cache = rknn.query('QUERY_ROPE_CACHE')
```

9.3 KVCache Controller

The controller maintains independent cache state for each attention configuration. In addition to manually generating control inputs, callers can set recognized RoPE/cache control inputs to `None` for automatic completion by `inference()`. See 9.3.4 for binding and mask polarity configuration.

9.3.1 Obtain KVCache Control Inputs

API	**kvcache_controller.generate_kvcache_control_tensors**
Description	Obtains the inputs for controlling the KVCache and the ID corresponding to the model's dynamic shape.
Parameters	**input_seq_len**: The number of tokens in the model input sequence.
Parameters	**system_prompt_len**: The number of tokens in the system prompt. Default value is 0.
Parameters	**history_seq_len**: History length for this inference. In paged mode, a value below cached `Ti` rolls the cache back; a nonzero rollback position must align to both `kvcache_group_size` and 32. Defaults to the controller's current state.
Parameters	**request_mask**: Per-request enable flags in paged mode, each 0 or 1. Accepts a scalar or `request_num` values; defaults to all 1s.
Return Value	Returns `(attention_inputs, dynamic_idx)`. The outer `attention_inputs` list follows input chunks. Within each chunk, control tensors for all attention configurations are flattened in configuration order. The single-attention return format remains compatible. `dynamic_idx` identifies the corresponding model dynamic shape.

In paged mode, each step supplies one `int32` control tensor per attention configuration, with shape `[request_num, 9]`. The columns are `Th, Tc, Ts, Tsr, longest_surviving_id, store_begin_id, Tr, Ti, request_mask`. Capacity and cache pointers are computed separately for each attention configuration; SlidingAttention and FullAttention must not share one control tensor. A single request supports chunked prefill. Multiple requests support decode only, with `Tc` of 0 or 1 per request. Requests with `request_mask=0` or `Tc=0` do not advance cache state.

The manual generation API advances the controller's recorded state. Callers must execute the returned chunks in order and handle inference failures. Fully manual RoPE and control inputs remain supported without automatic completion.

In paged mode, `Ts/Tsr` must remain unchanged after a system prompt takes effect. A nonzero rollback target must not be below the reserved system-prompt length `Ts`. Clear state and process the input again to change the system prompt.

The sample code is as follows:

```
attention_inputs, dynamic_idx =
rknn.kvcache_controller.generate_kvcache_control_tensors(input_seq_len)
```

9.3.2 Clear KVCache Status

API	kvcache_controller.clear_kvcache_status
Description	Clears controller state for all attention configurations so that the next inference starts with empty history. Does not release allocated cache pages.
Parameters	None.
Return Value	None.

The sample code is as follows:

```
rknn.kvcache_controller.clear_kvcache_status()
```

9.3.3 Roll back cache position

API	kvcache_controller.roll_back(roll_back_len)
Description	Subtracts a token count from the controller's current history so that the next step uses the earlier position for each attention configuration. The count must not exceed existing history or select positions already evicted from a circular cache. In paged mode, a nonzero target position must align to both `kvcache_group_size` and 32. All configurations are checked immediately; invalid requests preserve the original state.
Parameter	**roll_back_len**: A nonnegative integer or `request_num` nonnegative integers.
Return value	None.

```
rknn.kvcache_controller.roll_back(32)
```

9.3.4 Bind and Automatically Complete Control Inputs

API	kvcache_controller.bind_kvcache(mask_valid_value=1, mask_invalid_value=0)
Description	On the board, binds the current model's configured KV cache pages and configures valid-token and padding values in the current token mask. Missing RoPE/control inputs are generated from model metadata during inference. Call after loading the model and initializing runtime. This API does not provide cache switching.
Parameter	**mask_valid_value**: The mask value for valid tokens. Defaults to 1. Accepts a scalar or a dictionary keyed by each configuration's `mask_name`.
Parameter	**mask_invalid_value**: The mask value for padding. Defaults to 0 and must differ from the valid value. Accepts a scalar or the same kind of dictionary.
Return value	Returns 0 on success; invalid configuration raises an exception.

For each attention configuration, `inference()` reads the current mask input identified by `mask_name` and derives the valid token count `Tc`. The controller maintains history `Th`; the mask must describe only the current input. Automatic mode supports only `(B, N)` masks, with valid tokens followed by padding in each row. Higher-dimensional masks such as `(B, N, 1, N)`, left padding, gaps among valid tokens, or unconfigured mask values cause an error. A single

request requires at least one valid token. Multiple requests in paged mode support decode only, with 0 or 1 valid token per row. Mask polarity configuration does not change the control tensor's `request_mask` convention: `0=disabled, 1=enabled`.

Set input positions identified by model metadata `rope_cache_names` and `control_input_names` to `None` to generate these tensors automatically. All other model inputs must be provided explicitly. Follow the converted model's actual input order. Without an explicit binding call, the first automatic inference binds with the default `1=valid, 0=padding` values. Completion uses a copy of the call's input list, so callers can reuse their `None` placeholders.

Within one call, control inputs for all attention configurations must be either all `None` or all explicit. After manually generating control tensors, supply those tensors for that call rather than generating them again with `None`. RoPE inputs may independently be `None`; completion then uses positions and lengths from the explicit controls without advancing history again. Automatic controls commit state after successful KV writes. A failure with an uncertain write outcome invalidates the state; clear controller state and start again before continuing.

After reinitializing or replacing runtime, any nonzero cache history requires `clear_kvcache_status()` before rebinding. The controller preserves configured mask values; cache history from the previous runtime cannot be reused with the new runtime.

`SKIP_WRITE` does not advance cache state. Automatic controls support combined `DEFERRED_WRITE` calls with model IR `infer_mask=0`; phased calls with nonzero `infer_mask` require explicit control inputs.

Automatic completion does not currently support `accuracy_analysis=True`. For accuracy analysis, explicitly provide all RoPE/control tensors using the manual input path.

For ordinary text RoPE, `position_embeddings_host_storage=True` uses tables obtained from `QUERY_ROPE_CACHE`, selects positions using logical history and valid length, and pads to the current input shape. With `False`, complete RoPE tables are supplied and the operator selects positions using history and valid length. MRoPE paths that require additional position inputs still require callers to supply those inputs.

For a single-attention model in paged mode whose inputs are ordered as embedding, mask, logits index, two RoPE tables, and control tensor:

```
ret = rknn.kvcache_controller.bind_kvcache(mask_valid_value=1,
mask_invalid_value=0)
if ret != 0:
    raise RuntimeError('Bind KVCache controller failed')
inputs = [input_embeds, attention_mask, num_logits_to_keep, None, None, None]
outputs = rknn.inference(inputs, data_format=['nchw'] * len(inputs))
```

For models using `0=valid, 1=padding`, swap the binding values. Board cache page capacity remains managed by `alloc_kvcache_pages()` and `grow_kvcache_pages()`; callers can continue reserving capacity for the prompt and generation budget before inference.

10 ONNX-to-PyTorch Model Generation and Rebuild

The ONNX-to-PyTorch feature converts the intermediate model produced by RKNN build into a readable and editable PyTorch `nn.Module`. It can be used with the PyTorch Simulator and per-layer accuracy analysis, and can rebuild an RKNN model after model code, weights, or quantization parameters are modified. This feature is disabled by default. With `generate_torch_model=False`, the existing build/export workflow is unchanged.

10.1 Generate a PyTorch Model

Set `generate_torch_model=True` in `RKNN.config`, then run model loading, build, and `export_rknn` normally:

```
from rknn.api import RKNN

rknn = RKNN(verbose=True)
rknn.config(
    target_platform="rk182x",
    generate_torch_model=True,
)
rknn.load_onnx(model="model.onnx")
rknn.build(do_quantization=True, dataset="dataset.txt")
rknn.export_rknn("model.rknn")
rknn.release()
```

For a non-quantized model, set `do_quantization=False`; no `dataset` is required. The main artifacts are:

Artifact	Description
`<build_cache_dir>/torch_model/generated.py`	Editable PyTorch model source.
`<build_cache_dir>/torch_model/generated_weights.pth`	Model weights and quantization buffers.
`<build_cache_dir>/torch_model/model_generated.onnx`	Regenerated ONNX used by `rebuild`.
`<build_cache_dir>/torch_model/input_preprocess.json`	Input preprocessing configuration.
`<build_cache_dir>/torch_model/meta.json`	Model and original config metadata required by `rebuild`.
`<model>_quant_tab.json`	Quantization manifest for a quantized model.

These files form one artifact set. When copying or archiving a model, keep the complete `torch_model` directory, `quant_tab.json`, and every referenced PTH file. Do not mix files from different builds.

For a LoRA model loaded through `load_lora`, the generated artifacts also preserve the LoRA weights, pattern/group metadata, and enabled state. Use `set_lora_state()` on the generated PyTorch model to switch LoRA state; `rebuild` continues to use the same LoRA metadata.

10.2 Use the PyTorch Simulator

Pass the generated artifact directory to `init_runtime`:

```
from rknn.api import RKNN

rknn = RKNN(verbose=True)
rknn.config(target_platform="rk182x")
rknn.init_runtime(
    target="simulator",
    simulator_backend="pytorch",
    pytorch_artifact_dir="tmp/torch_model",
    pytorch_quant_json="model_quant_tab.json",
)
outputs = rknn.inference(inputs=[input_data])
rknn.release()
```

For a quantized model, explicitly pass its matching `pytorch_quant_json`. A floating-point model does not need this parameter. Enable per-layer accuracy analysis through `inference` as usual:

```
outputs = rknn.inference(
    inputs=[input_data],
    accuracy_analysis=True,
    output_dir="./accuracy_analysis",
)
```

`simulator_backend="pytorch"` only selects the software simulator backend; it does not change how the final RKNN model executes on a device.

10.3 Load and Edit a Generated Model

Use only the public loading API when running generated artifacts as a regular PyTorch model:

API	Description
`load_generated_module(py_path)`	Loads a generated module from an explicit `generated.py` path and returns the Python module containing `Model`.
`load_model_from_weight_store(model_class, base_path, map_location="cpu", strict=True, mmap=True)`	Loads `model_class` from the weight store corresponding to `base_path` and returns `(model, state_dict)`. Pass a file prefix such as `tmp/torch_model/generated`; do not append `_weights.pth`.

```python
import torch

from rknn.api.onnx_torch_codegen import (
    load_generated_module,
    load_model_from_weight_store,
)

artifact_dir = "tmp/torch_model"
module = load_generated_module(f"{artifact_dir}/generated.py")
model, _ = load_model_from_weight_store(
    module.Model,
    f"{artifact_dir}/generated",
    map_location="cpu",
)
model.eval()

with torch.inference_mode():
    outputs = model.inference(torch.from_numpy(input_data))
```

Do not import Toolkit implementation modules directly; those paths are not public APIs. Load artifacts only through the public helpers above. You may edit operators or `forward` in `generated.py`, or modify the weight store, while preserving valid input, output, and shape contracts. A generated model requires the matching RKNN3-Toolkit version and is not a fully standalone PyTorch distribution.

10.4 Rebuild RKNN from Generated Artifacts

After editing generated source, weights, or quantization parameters, rebuild from the original build cache:

```python
from rknn.api import RKNN

rknn = RKNN(verbose=True)
rknn.rebuild(
    build_cache_dir="tmp",
    quant_json="model_quant_tab.json",
    dataset="dataset.txt",
)
rknn.export_rknn("model_rebuilt.rknn")
rknn.release()
```

- If only compatible quantization parameters are replaced and graph structure is unchanged, calibration is not required and `dataset` can be omitted.
- If a quantized graph is structurally modified, provide `dataset`.
- If `quant_json` is omitted, the original build-cache quantization parameters are used.
- `rebuild` works in a staging directory and updates the original cache only after success.
- Validate final deployment behavior with the rebuilt RKNN model on the target device.

10.5 Quantization Parameter Storage and Replacement

For quantization tensors used by forward QDQ and rebuild, `generated.py` declares only each buffer's name, shape, and dtype. It does not store actual weight/activation scales, zero points,

min/max values, or quantized integer weights. Actual values are stored in `generated_weights.pth`, and `quant_tab.json` maps ONNX tensors to PTH buffer keys. This allows quantization values with the same schema to be replaced without editing generated source, while fixed shapes and dtypes reject incompatible configurations. Input preprocessing configuration such as mean/std or fused-mean/std is still recorded in generated-model metadata; it is not an in-graph QDQ buffer replaceable through the quant manifest.

Each tensor entry in `quant_tab.json` records `dtype`, `qmethod`, `qtype`, and PTH buffer keys. The top-level `_torch_weights` path points to the PTH file and is resolved relative to the JSON file. Some models also contain `_torch_quant_params`; keep its companion PTH file as well.

If two models have identical graph structure, tensor entries, buffer keys, parameter shapes, and quantization schema, copy the original manifest, update `_torch_weights` to the new PTH path, and pass that manifest to `rebuild`:

```
rknn.rebuild(
    build_cache_dir="model_b/tmp",
    quant_json="custom_quant_tab.json",
)
```

A bare PTH file cannot be passed as `quant_json`. If changing quantization dtype, method, or group size changes a buffer shape or dtype, regenerate the complete Torch artifact set instead of replacing only the PTH file.

10.6 Example and Notes

The quantization-parameter replacement example is in `examples/functions/rebuild_with_custom_qparam/`. It builds two calibrations and rebuilds one model's cache with the other model's compatible manifest/PTH:

```
cd examples/functions/rebuild_with_custom_qparam
python3 test.py
python3 test.py --device-id <device-id>
```

Do not mix `generated.py`, PTH, manifest, or `meta.json` files from different builds. When modifying scale or zero point, keep min/max values and quantized integer weights consistent. Provide a new quantization dataset after changing model inputs, outputs, or graph structure.

11 RKQuantizer

RKQuantizer is an independent quantization component provided by RKNN3-Toolkit. It performs post-training quantization (Post-Training Quantization) on the Linear layer weights of a PyTorch floating-point model before the model is exported to ONNX. RKQuantizer works directly on a `torch.nn.Module` and returns a fake-quantized model after quantization. The user then exports it to ONNX, and finally loads it through the `rknn.load_llm` interface and calls `rknn.build` to complete the RKNN model conversion. The typical workflow is as follows:

```
Floating-point PyTorch model ──RKQuantizer──▶ fake-quant PyTorch model ──export
ONNX──▶ rknn.load_llm ──▶ rknn.build
```

Before use, import RKQuantizer from `rknn.quantization.api`:

```
from rknn.quantization.api import RKQuantizer
```

RKQuantizer usage is divided into five parts: API description, calibration dataset preparation, AWQ configuration, mixed-precision quantization, and GRQ memory usage. See sections 11.1, 11.2, 11.3, 11.4, and 11.5 respectively.

Supported Model Limitations

RKQuantizer does not support arbitrary PyTorch models. The following prerequisites must be met before use:

1. **Model architecture**: Currently, mainly Transformer-architecture models are supported (e.g., the language/vision parts of various LLMs and VLMs).
2. **Quantization target**: RKQuantizer only quantizes `torch.nn.Linear` layers in the model. Other operator types (e.g., convolution, Embedding, LayerNorm) are not quantized and keep their original precision.
3. **GRQ scope**: The GRQ algorithm (`quantized_algorithm='grq'`) automatically searches the passed-in model for a `torch.nn.ModuleList` (falling back to `torch.nn.Sequential` if none is found) as the transformer block sequence; if neither exists, a "No nn.ModuleList or nn.Sequential found in the passed model, the grq quantization algorithm can not be used" error is raised. Therefore, the model must organize its stacked blocks as a `ModuleList` or `Sequential`. Note that if there are multiple `ModuleList`s in the model, the longest one (by number of elements) is selected as the target for quantization. The scope of AWQ is also defined by this block sequence.
4. **Device requirement**: The GRQ algorithm is recommended to run under CUDA or MPS (it takes a very long time on CPU). The normal algorithm can run under CUDA, MPS, or CPU environments.

11.1 API Description

The following tables describe the RKQuantizer APIs.

API	RKQuantizer
Description	Initializes an RKQuantizer object.
Parameter	**verbose**: Whether to print verbose log information. Valid values are True, False, and 'debug'. Default is False.
	verbose_file: Log file path. When this parameter is set and verbose is True, log information is also written to this file. If the directory does not exist, this parameter is ignored. Default is None.
Return Value	RKQuantizer object.

API	load_model
Description	Loads the PyTorch floating-point model to be quantized and its corresponding tokenizer, etc., into the quantization tool.
Parameter	**model**: The PyTorch model to be quantized (`torch.nn.Module`). This parameter specifies the sub-model that actually participates in quantization, and is very important. RKQuantizer only quantizes the `model` that is passed in. When using the **grq quantization algorithm**, **the passed-in model should preferably contain only one main container of `nn.ModuleList` / `nn.Sequential` (the `nn.ModuleList` / `nn.Sequential` inside the main container has no effect)**: if more than one is detected, RKQuantizer only emits a warning and proceeds, defaulting to the longest one (by number of elements) as the quantization target (this usually means an overly large sub-model was passed in, for example, passing in the entire VLM, where vision and llm each have their own block list). If the quantized result is not what you expect, follow the warning and pass in the correct sub-module to be quantized (such as `model.model` / `model.visual`). The specific value typically falls into two cases: **- Pure LLM model**: pass in the loaded CausalLM model itself (`model=model`). **- VLM (multimodal) model**: a VLM consists of a vision part and an llm part, which must be quantized separately — **do not pass in the entire VLM model**. Instead, pass in the corresponding sub-module according to the quantization target: · When quantizing the vision part, pass in the vision sub-module (e.g., `model.visual`, `model.model.vision_model`, `model.model.vision_tower`, depending on the model structure); · When quantizing the llm part, pass in the language model sub-module (e.g., `model.model`, `model.model.text_model`, `model.model.language_model`, `model.language_model`, depending on the model structure). The exact names of the corresponding sub-modules for each model can be confirmed by printing the model structure.

API	load_model
	model_lora: Path to a LoRA adapter in HuggingFace format (a LoRA adapter directory). Default is None. When set, RKQuantizer reads `adapter_model.safetensors` (LoRA weights) and `adapter_config.json` (LoRA config) from that directory, casts the `lora_A` / `lora_B` weights to the same dtype as the model, and attaches them to the corresponding Linear layers during quantization by matching the Linear layer name suffix (`<layer_name>.lora_A.weight`, `<layer_name>.lora_B.weight`). The scale factor is `scale = lora_alpha / rank`, where `lora_alpha` is read from `adapter_config.json` and `rank` is the output dimension of `lora_B`. Key matching is compatible with both the `base_model.model.` prefix (the default PEFT export format) and no prefix, but each suffix must match a unique key in the LoRA weights, otherwise an error is raised. **Note**: (1) This feature is mainly used with `quantized_algorithm='grq'` so that GRQ is aware of the LoRA adaptation; (2) When `model_lora` is provided, the AWQ stage is skipped automatically (AWQ and LoRA cannot be used together) - even if `awq_config` is set, AWQ will not run.
	tokenizer: The tokenizer paired with the model, used to encode text when the calibration data is in "text-only JSON mode". It can be set to None when the calibration data is in "hook + pickle mode".
	device: The device on which quantization is executed. 'cuda', 'mps', 'cpu', or 'auto' are supported. Default is 'cuda'. When set to 'auto', RKQuantizer selects 'cuda' > 'mps' > 'cpu' according to the available PyTorch backends, not the operating system name. When using 'grq', 'cuda' or an available 'mps' device is recommended; CPU execution emits a slow-speed warning.
	system_prompt: The system prompt (a string). Only effective for calibration data in "text-only JSON mode". When either `system_prompt` or `tools` is not None, the `input+target` text of each sample is assembled into a conversation through the tokenizer's chat template, making the calibration input distribution closer to the real deployment scenario; when both are None, the sample text is encoded as-is without applying the chat template. When `system_prompt` is not None, a system message is inserted into the conversation; otherwise only user messages are constructed. Default is None. This parameter has no effect under "hook + pickle mode" calibration data.
	tools: A list of tool definitions, passed through to the `tools` parameter of `tokenizer.apply_chat_template`. If either `tools` or `system_prompt` is not None, the chat template is applied, used to construct prompts with tool calling (function calling); `tools` can be passed alone (without `system_prompt`) to match a deployment scenario that has no system prompt but enables function calling. Default is None.
	system_role: The role name used for the system message when applying the chat template (only used when `system_prompt` is not None). Default is 'system'.
	user_role: The role name used for the user message when applying the chat template. Default is 'user'.
Return Value	0: Load succeeded. -1: Load failed.
API	**quantize**
Description	Quantizes the loaded PyTorch model and returns the fake-quantized PyTorch model. **Note: this API returns the quantized model object, not a 0/-1 status code.**

API	quantize
Parameter	**quantized_dtype**: Quantization dtype. Currently w4a16, w6a16, and w8a16 are supported. Default is w4a16 (recommended). w4a16 means 4-bit asymmetric weight quantization and 16-bit floating-point activation precision.
	quantized_method: Weight quantization method. Currently only 'group32' is supported. Default is 'group32'. 'group32' means that, on top of per-channel quantization, each output channel is further divided into multiple small groups of 32 along the input-channel dimension, with one set of quantization parameters per group. This usually provides higher accuracy.
	quantized_algorithm: Quantization algorithm. Currently 'grq' and 'normal' are supported. Default is 'grq'. GRQ fine-tunes the model weights on top of quantization, providing higher accuracy. It is recommended to run under CUDA or MPS and requires a calibration dataset; CPU execution is supported but very slow and emits a warning. The normal algorithm does not require a calibration dataset.
	dataset: Calibration dataset path, used for quantization parameter statistics and GRQ fine-tuning. Usually points to a JSON file (containing several calibration samples). The calibration dataset has strict rules; see section 11.2 for details. If `dataset` is set to None, `quantized_algorithm` will be forced to be replaced with the normal quantization algorithm.
	awq_config: AWQ quantization configuration, of dictionary type. When configured, AWQ runs before GRQ; when None, AWQ is skipped. The dictionary describes the transformer block structure of the model and contains submodule names such as BLOCKNAME (block module name, e.g., 'Qwen3DecoderLayer'), ATTN_NORM/ATTN_Q/ATTN_K/ATTN_V/ATTN_QKV/ATTN_KV/ATTN_OUT, and FFN_NORM/FFN_UP/FFN_GATE/FFN_DOWN. Default is None. Additionally, when LoRA weights are provided to `load_model` (via the `model_lora` parameter), AWQ is skipped automatically (AWQ and LoRA cannot be used together). See section 11.3 for details on AWQ configuration.
	auto_hybrid_rate: Automatic mixed-precision ratio. The valid range is a float in [0, 1). Default is 0 (disabled). When enabled, a ratio of sensitive Linear layers is quantized with the higher-precision w8a16 dtype to improve accuracy. See section 11.4 for details on mixed-precision quantization. **Note: this parameter cannot be used together with layer_quant_config.**
	layer_quant_config: Per-layer quantization dtype configuration, supporting a dictionary or a JSON file path. The format is `{layer_pattern: "w8a16"}`, where `layer_pattern` is a shell-style wildcard pattern for matching Linear layer names (e.g., `model.layers.*.mlp.down_proj`); matched layers are quantized with the specified dtype. Currently "w6a16" or "w8a16" are supported. This parameter cannot be used together with auto_hybrid_rate. Default is None. See section 11.4 for details on mixed-precision quantization.
Return Value	The fake-quantized `torch.nn.Module` model, corresponding to the model passed to the `model` parameter of load_model. Failures are raised through RKNNLog.

API	export_op_quantized_dtype
Description	When mixed-precision quantization is enabled (via layer_quant_config or auto_hybrid_rate), exports the actual quantization dtype used by each layer to a JSON file, so that it can be aligned through the `op_quantized_dtype` parameter in the subsequent `rknn.config`. When mixed-precision quantization is not enabled, this API does not need to be called; calling it will print a warning and return directly.
Parameter	**onnx_path**: Path of the exported ONNX model, used to map PyTorch layer names to ONNX operators.
	op_dtype_path: Path of the exported quantization dtype JSON file. Default is 'layer_bit.json'.
Return Value	None.

Example:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from rknn.quantization.api import RKQuantizer

# 1. Load the floating-point model and tokenizer
model_path = 'Qwen/Qwen3-1.7B'
model = AutoModelForCausalLM.from_pretrained(model_path, torch_dtype=torch.float16,
trust_remote_code=True)
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)

# 2. Initialize the quantization tool and load the model
QuantTool = RKQuantizer(verbose=True)
ret = QuantTool.load_model(model=model, tokenizer=tokenizer, device='cuda')
if ret != 0:
    print('Load model failed!')
    exit(ret)

# 2.1 (Optional) If the model is deployed with a system prompt / tool calling,
#     the calibration input can apply the same chat template so that the
#     calibration distribution matches the real deployment (only effective for
#     "text-only JSON mode" calibration data):
# system_prompt = "You are a helpful assistant."
# tools = [...]  # List of tool definitions, passed through to
tokenizer.apply_chat_template
# ret = QuantTool.load_model(model=model, tokenizer=tokenizer, device='cuda',
#                            system_prompt=system_prompt, tools=tools,
#                            system_role='system', user_role='user')

# 3. Quantize (GRQ + w4a16 + group32) and return the fake-quantized model
model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm='grq',
                           dataset='./dataset.json')
model = model.cpu()

# 4. Export the quantized model to ONNX (attention must be switched back to eager
for export)
#    export_to_onnx(model, './Qwen3-1.7B.onnx', ...)
```

If the deployment model has a LoRA adapter (HuggingFace format, e.g., an adapter trained with PEFT), you can pass the LoRA weight directory via `model_lora` when calling `load_model`, so that GRQ performs quantization fine-tuning on the effective weights of "base model + LoRA". The passed-in `model` should be the base model with LoRA not yet merged, and AWQ will be skipped automatically in this case:

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from rknn.quantization.api import RKQuantizer

# Base model and LoRA adapter (HuggingFace format; the directory must contain
# adapter_model.safetensors and adapter_config.json)
model_path = 'Qwen/Qwen3-1.7B'
lora_path = './qwen3_lora_adapter'

model = AutoModelForCausalLM.from_pretrained(model_path, torch_dtype=torch.float16,
trust_remote_code=True)
tokenizer = AutoTokenizer.from_pretrained(model_path, trust_remote_code=True)

QuantTool = RKQuantizer(verbose=True)
# Pass the LoRA weight directory via model_lora; GRQ will quantize on base + LoRA
effective weights
ret = QuantTool.load_model(model=model, model_lora=lora_path, tokenizer=tokenizer,
device='cuda')
if ret != 0:
    print('Load model failed!')
    exit(ret)

# When LoRA is provided, AWQ is skipped automatically (AWQ and LoRA cannot be used
together); only GRQ runs
model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm='grq',
                           dataset='./dataset.json')
model = model.cpu()
```

For VLM (multimodal) models, the vision part and the llm part must be quantized separately. Each call to `load_model` should pass in only the corresponding sub-module rather than the entire VLM. Taking Qwen2.5-VL as an example:

```python
from transformers import Qwen2_5_VLForConditionalGeneration, AutoProcessor

model_path = 'Qwen/Qwen2.5-VL-3B-Instruct'
vlm = Qwen2_5_VLForConditionalGeneration.from_pretrained(model_path,
torch_dtype=torch.float16, trust_remote_code=True)

QuantTool = RKQuantizer(verbose=True)

# Quantize the vision part: pass in the vision sub-module; tokenizer can be set to
None depending on the calibration data
ret = QuantTool.load_model(model=vlm.visual, tokenizer=None, device='cuda')
# ... call quantize to export the vision ONNX ...

# Quantize the llm part: pass in the language model sub-module; tokenizer can be
set to None depending on the calibration data
ret = QuantTool.load_model(model=vlm.model, tokenizer=None, device='cuda')
# ... call quantize to export the llm ONNX ...
```

The sub-module naming of different VLMs varies slightly. Common correspondences are as follows (subject to the actual model structure printed):

Quantization target	Common sub-module path examples
vision part	`model.visual`, `model.model.vision_model`, `model.model.vision_tower`, `model.vision_model`
llm part	`model.model`, `model.model.text_model`, `model.model.language_model`, `model.language_model`

11.2 Calibration Dataset Preparation

When the `quantize` API performs GRQ quantization (as well as AWQ and automatic mixed precision), it requires a calibration dataset (the `dataset` parameter) to compute the activation distribution of each layer and fine-tune the weights. The calibration dataset is provided as a **JSON file**. The file content is a list, and each element is one sample. RKQuantizer reads each sample one by one and constructs the model inputs. Currently two sample formats are supported, and users can choose according to the model type.

Mode 1: hook + pickle (general mode, recommended)

The most general way is to directly capture the real inputs of the **model to be quantized** (i.e., the sub-module passed to `load_model`, such as `model` for a pure LLM, or `model.model` / `model.visual` for a VLM). Use PyTorch's hook mechanism to capture the inputs of this sub-module during the forward pass of the floating-point model, serialize and save them with pickle, and then build a JSON index file. This mode applies to all models, including multimodal models (VLMs), speech models, and other models with non-text-only inputs.

> **Note**: The hook captures the input of the **entire model to be quantized (i.e., the sub-module passed to `load_model`)**, not the input of the first transformer block inside that model. During quantization, RKQuantizer feeds this input as a whole to the model to be quantized, and then computes the activation layer by layer inside it. Therefore, the capture point must be at the outermost entry of the model to be quantized.

Each sample in the JSON file is a dictionary containing only a `sample` field, whose value is the relative path of a pickle file (relative to the directory where the JSON file is located). The pickle file content is a dictionary containing two keys, `args` (positional arguments) and `kwargs` (keyword arguments), corresponding to the positional and keyword inputs of the forward call of the model to be quantized. The format is as follows:

```
[
  {"sample": "model_inputs/sample_0"},
  {"sample": "model_inputs/sample_1"}
]
```

The standard way to obtain this data is: on the floating-point model to be quantized, register a `register_forward_pre_hook` on the model to be quantized (i.e., the sub-module passed to `load_model`) (with `with_kwargs=True` enabled to also capture keyword arguments), save the inputs to a list in the hook and raise an exception to terminate the forward early (to avoid executing a full inference), and then pickle the captured inputs to disk. To avoid duplicating hook boilerplate code across demos, rknn3-model-zoo wraps this mechanism into a public utility `py_utils/cali_data.py`, providing the `capture_module_input(hook_module, run, *args, **kwargs)` function: it packages "register forward pre hook → call the model → capture `(args, kwargs)` → raise `StopForward` to short-circuit the forward → remove the hook" into a single line, so the caller does not need to deal with hook details. Its internal hook raises `StopForward` to terminate the forward of the captured sub-module, so even if `model.generate` is passed in, only one forward pass is run before it is interrupted, and no tokens are actually generated. Example:

```python
import os
import json
import pickle
import torch
from transformers import AutoProcessor
# Import the public hook capture utility (adjust the path according to your project
structure)
from py_utils.cali_data import capture_module_input

model_path = 'Qwen/Qwen2.5-VL-3B-Instruct'
model = ...                      # Load the floating-point model; torch_dtype must
match quantization
processor = AutoProcessor.from_pretrained(model_path)

# Forward each sample and capture the input entering the sub-module to be quantized
in one line
info = []
datasets = json.load(open('./dataset.json', 'r'))   # Raw annotated data
(containing images/questions, etc.)
for idx, data in enumerate(datasets):
    inputs = processor(...)                # Construct full inputs according to
the model requirements
    inputs = inputs.to(model.device)
    with torch.inference_mode():
        # Pass the model to be quantized itself as hook_module (i.e., the sub-
module passed to load_model;
        # here we quantize the vision part, so model.visual is passed).
        # The captured input is the input at the outermost entry of this sub-
module,
        # not the input of the first transformer block inside it.
        # Pass the model call to trigger as run (wrapped in a lambda; may include
prepare_inputs_embeds, etc.)
        temp = capture_module_input(
            model.visual,
            lambda: model(**inputs)
        )
        # temp is {"args": (...), "kwargs": {...}}; None if the input does not flow
through this sub-module

    # Pickle the captured input to disk and record it in the index JSON
    sample_name = 'sample_{}'.format(idx)
    os.makedirs('./model_inputs', exist_ok=True)
    with open(os.path.join('./model_inputs', sample_name), 'wb') as f:
        pickle.dump(temp, f)
    info.append({"sample": "model_inputs/{}".format(sample_name)})

with open('./cali_data.json', 'w', encoding='utf-8') as f:
    json.dump(info, f, indent=2, ensure_ascii=False)
```

When adapting to a new model, copy the `make_calidata.py` of the closest demo and change only three things: the model loading method, `hook_module` (the sub-module to be quantized, such as `model.model` / `model.visual`), and `build_inputs` (constructing the model inputs). The capture call can directly reuse `capture_module_input`. `run` can also be passed directly as a model method with arguments forwarded: `capture_module_input(model.model, model.generate, **inputs, max_new_tokens=128)`.

Notes:

1. The first parameter of `capture_module_input` must be the **model to be quantized itself** (i.e., the sub-module passed when calling `load_model`, such as `model` for a pure LLM, or `model.model` / `model.model.text_model` / `model.visual` for a VLM). The captured input is the input at the **outermost entry of this sub-module**, not the input of the first transformer block inside it. During quantization, RKQuantizer feeds this input as a whole to this sub-module and then computes the activation layer by layer inside it, so the capture point must be the same object passed to `load_model`.
2. The `torch_dtype` used when loading the floating-point model should be consistent with that used for quantization; otherwise the numerical precision of the captured inputs may not match the quantization stage.
3. The `StopForward` raised by the internal hook interrupts the forward pass. The function handles it internally with `try/except`, so the caller does not need to wrap it again; the hook is always removed before the function returns and will not leak into subsequent calls.
4. If you do not use the public utility and implement the hook yourself, you must `raise` an exception in the pre hook to terminate the forward pass, and manage the registration and removal of the hook yourself.
5. The recommended number of calibration samples is 20 to 128. Samples should cover the input distribution of the actual deployment scenario as much as possible.

Mode 2: text-only JSON (only for text-only LLMs)

For LLM models whose inputs are only text, a simpler mode can be used: each sample in the JSON file is a dictionary containing two keys, `input` and `target`, and RKQuantizer internally uses `tokenizer(input + target)` to encode it into model inputs. This mode does not require loading the floating-point model or using hooks, and has the lowest preparation cost. The format is as follows:

```
[
  {
    "input": "What can cysteine and tyrosine be converted from in the human body?
\nA. Cystine and methionine\nB. Methionine and phenylalanine",
    "target": "\nAnswer:\n B"
  },
  {
    "input": "Natalia sold clips to 48 of her friends in April, and then she sold
half as many in May. How many altogether?",
    "target": ""
  }
]
```

Here `input` is the prompt text and `target` is the expected output text (which can be an empty string). The two are concatenated and sent to the tokenizer as a whole; `target` can include answers or chain-of-thought context to make the calibration distribution closer to the real generation scenario. Example:

```
import json

# Write directly, or curate from an existing dataset, and save as JSON
samples = [
    {"input": "Briefly describe Newton's first law.", "target": "\nAnswer:\n
Newton's first law states..."},
    {"input": "What is 1+1?", "target": "\nAnswer:\n 2"},
]
with open('./cali_data.json', 'w', encoding='utf-8') as f:
    json.dump(samples, f, ensure_ascii=False, indent=2)
```

Notes:

1. This mode only applies to text-only LLMs. For multimodal (VLM), speech, or other models with non-text inputs, Mode 1 must be used.

Selection of calibration data

The quality of the calibration data greatly affects the accuracy of the quantized model. The core principle when selecting data is: **use diverse data from the real application scenario** so that the calibration data distribution is as close as possible to the actual input distribution after the model is deployed. Specific recommendations are as follows:

1. **Cover real scenarios**: Calibration samples should come from the real inputs of the actual deployment scenario of the model (the actual system prompt, function-tools (**very important for agent tasks**), etc., which can be loaded via the load_model API), rather than arbitrary general corpora.
2. **Maintain diversity**: Samples should be diverse and avoid highly homogeneous content, so that the quantization parameters can adapt to different input distributions.
3. **Distinguish model types**:
4. For **open-source general base models**, the impact of calibration data is relatively small, and using general corpora usually yields acceptable quantization accuracy;
5. For **models fine-tuned on vertical-domain tasks**, the impact of calibration data is very large. For example, a model fine-tuned on a vertical domain pays special attention to the system prompt and structured data (such as JSON, tables, specific-format instructions, etc.). If such data is not fed in during the quantization (GRQ fine-tuning) stage, the model's capability on the vertical task will be damaged, leading to a noticeable degradation on vertical tasks after quantization. Therefore, vertical fine-tuned models must use a system prompt and structured data consistent with the actual vertical task as calibration samples.

11.3 AWQ Configuration

AWQ (Activation-aware Weight Quantization) is a quantization algorithm that rescales and corrects weights based on the activation distribution, which can further reduce accuracy loss under low-bit quantization (such as w4a16). In RKQuantizer, when `quantized_algorithm='grq'` of the `quantize` API and `awq_config` is not None, AWQ runs before GRQ; when `awq_config` is None, AWQ is skipped.

`awq_config` is a dictionary that describes the internal structure of the model's transformer blocks, enabling the AWQ algorithm to locate the attention-related Linear layers, FFN-related Linear layers, and the front/back LayerNorm modules in each block. The fields are described as follows:

Field	Description
BLOCKNAME	Block module name, used to identify the model's transformer blocks (e.g., 'Qwen2DecoderLayer', 'LlamaDecoderLayer'). AWQ corrects layer by layer on a per-block basis. **This field is required.**
ATTN_NORM	LayerNorm submodule name before the attention module (e.g., 'input_layernorm'), used to locate the attention input source.
ATTN_Q	Attention Q projection Linear submodule name (e.g., 'self_attn.q_proj'). Fill this when Q/K/V are separate projections; leave it as an empty string for merged projection.
ATTN_K	Attention K projection Linear submodule name (e.g., 'self_attn.k_proj'). Same usage as ATTN_Q.
ATTN_V	Attention V projection Linear submodule name (e.g., 'self_attn.v_proj'). Same usage as ATTN_Q.
ATTN_QKV	Attention merged QKV projection Linear submodule name (e.g., 'attn.qkv'). Fill this when Q/K/V are merged into a single Linear, and leave ATTN_Q/ATTN_K/ATTN_V empty.
ATTN_KV	Attention merged KV projection Linear submodule name. Fill this when K/V are merged (and Q is separate); leave the others empty.
ATTN_OUT	Attention output projection Linear submodule name (e.g., 'self_attn.o_proj', 'attn.proj').
FFN_NORM	LayerNorm submodule name before the FFN module (e.g., 'post_attention_layernorm'), used to locate the FFN input source.
FFN_UP	FFN up projection Linear submodule name (e.g., 'mlp.up_proj'). Fill this for FFN without a gate structure; leave it empty for gated FFN.
FFN_GATE	FFN gate projection Linear submodule name (e.g., 'mlp.gate_proj'). Fill this for gated FFN (e.g., LLaMA-style SwiGLU), used to locate the FFN input source.
FFN_DOWN	FFN down projection Linear submodule name (e.g., 'mlp.down_proj', 'mlp.linear_fc2').

Notes:

1. There are three common attention projection structures. Choose one according to the actual model and leave the others empty:
   - Separate Q/K/V projections: fill ATTN_Q/ATTN_K/ATTN_V, and leave ATTN_QKV/ATTN_KV empty;
   - Merged QKV projection: fill ATTN_QKV, and leave ATTN_Q/ATTN_K/ATTN_V/ATTN_KV empty;
   - Merged KV projection (Q separate): fill ATTN_Q and ATTN_KV, and leave the others empty.

2. The FFN fields must also be chosen according to the structure: for gated FFN (e.g., SwiGLU), fill FFN_GATE/FFN_DOWN and leave FFN_UP empty; for non-gated FFN, fill FFN_UP/FFN_DOWN and leave FFN_GATE empty.
3. Each field value is a relative path of a submodule inside a block (relative to the block module), consistent with the naming in the model's `named_modules()`. It can be obtained by printing the model structure.
4. `awq_config` takes effect only when `quantized_algorithm='grq'` and the configuration is not None.
5. AWQ can be used together with mixed-precision quantization (`layer_quant_config` / `auto_hybrid_rate`); they do not affect each other.
6. **Not every model can use AWQ.** AWQ reduces quantization error by rescaling and correcting weights, and its correction logic requires that the preceding operators of the key Linear layers in a transformer block (attention Q/K/V/QKV/KV/out, FFN up/gate/down) be one of LayerNorm or Linear. When the model structure does not satisfy this prerequisite (for example, a key Linear is preceded by another custom operator, or the block structure cannot be correctly identified by `BLOCKNAME`), an exception will be thrown internally. Therefore, before using AWQ, you should first confirm that the model structure can be correctly described by `awq_config`; for models that do not support AWQ, set `awq_config=None` to use GRQ quantization only. **Currently Qwen3.5 and Gemma4 models cannot use AWQ.**

Example configuration for LLM models with separate Q/K/V projections (such as Qwen2.5, LLaMA series):

```
awq_config = {
    "BLOCKNAME": "Qwen2DecoderLayer",
    "ATTN_NORM": "input_layernorm",
    "ATTN_Q": "self_attn.q_proj",
    "ATTN_K": "self_attn.k_proj",
    "ATTN_V": "self_attn.v_proj",
    "ATTN_QKV": "",
    "ATTN_KV": "",
    "ATTN_OUT": "self_attn.o_proj",
    "FFN_NORM": "post_attention_layernorm",
    "FFN_UP": "mlp.up_proj",
    "FFN_GATE": "mlp.gate_proj",
    "FFN_DOWN": "mlp.down_proj"
}

model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm='grq',
                           dataset='./dataset.json',
                           awq_config=awq_config)
```

Example configuration for vision models with merged QKV projection (such as the vision part of Qwen3-VL, Qwen2.5-VL):

```
awq_config = {
    "BLOCKNAME": "Qwen3VLVisionBlock",
    "ATTN_NORM": "norm1",
    "ATTN_Q": "",
    "ATTN_K": "",
    "ATTN_V": "",
    "ATTN_QKV": "attn.qkv",
    "ATTN_KV": "",
    "ATTN_OUT": "attn.proj",
    "FFN_NORM": "norm2",
    "FFN_UP": "",
    "FFN_GATE": "mlp.linear_fc1",
    "FFN_DOWN": "mlp.linear_fc2"
}

model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm='grq',
                           dataset='./dataset.json',
                           awq_config=awq_config)
```

About the accuracy gain of AWQ

According to actual measurements, under the w4a16 + group32 setting, the accuracy gain brought by enabling AWQ (i.e., `awq_config` is not None) is usually not obvious, and in some cases there may even be a slight accuracy degradation. Therefore, AWQ is not a mandatory option enabled by default. It is recommended to follow this principle: **prioritize using GRQ quantization only, and consider enabling AWQ only when the accuracy after GRQ quantization still does not meet the requirements**, and decide whether to keep the AWQ result by comparing the measured accuracy.

11.4 Mixed-Precision Quantization

When the whole model uses low-bit quantization (such as w4a16), some quantization-sensitive layers (such as v_proj in attention, down_proj in FFN, or specific-structure linear-attention layers) may have obvious accuracy loss. Mixed-precision quantization allows these sensitive layers to be kept at higher precision, while the remaining layers are still quantized according to the type specified by `quantized_dtype`, thus striking a balance between accuracy and model size/accuracy. RKQuantizer provides two mixed-precision quantization modes: manually specifying layers (`layer_quant_config`) and automatically selecting by ratio (`auto_hybrid_rate`).

Mode 1: Manually specify layers (layer_quant_config)

Users can explicitly specify which layers use w6a16 or w8a16. `layer_quant_config` can be a dictionary or a JSON file path. The format is `{layer_pattern: "w8a16"}`, where `layer_pattern` is a shell-style wildcard pattern for matching Linear layer names and supports the `*` wildcard (used to match the same-named sub-layer across all transformer blocks). The matched layer will use the quantization type specified in the configuration, while unmatched layers are quantized according to `quantized_dtype`. Currently `layer_quant_config` only supports the value "w8a16" or "w6a16" and cannot be used together with `auto_hybrid_rate`.

Taking the Qwen3.5 model as an example, its `layer_quant_config.json` content is as follows, specifying the linear-attention-related qkv/b/a/out_proj layers as well as v_proj and down_proj layers as w8a16 (or w6a16):

```
{
  "model.layers.*.linear_attn.in_proj_qkv": "w8a16",
  "model.layers.*.linear_attn.in_proj_b": "w8a16",
  "model.layers.*.linear_attn.in_proj_a": "w8a16",
  "model.layers.*.linear_attn.out_proj": "w8a16",
  "model.layers.*.self_attn.v_proj": "w8a16",
  "model.layers.*.mlp.down_proj": "w8a16"
}
```

The corresponding quantization call is as follows:

```
quant_model = QuantTool.quantize(quantized_dtype="w4a16",
                                 quantized_method="group32",
                                 quantized_algorithm='grq',
                                 dataset='./dataset.json',
                                 awq_config=None,
                                 layer_quant_config="layer_quant_config.json",
                                 auto_hybrid_rate=0.)
```

Mode 2: Automatically select by ratio (auto_hybrid_rate)

There is no need to specify each layer. RKQuantizer automatically evaluates the sensitivity of each layer to quantization based on the calibration data, and keeps the most sensitive Linear layers at w8a16 according to the specified ratio. `auto_hybrid_rate` is a float in [0, 1), representing the ratio of Linear layers set to w8a16. For example, 0.2 means about 20% of Linear layers are set to w8a16, while the remaining layers are quantized according to

`quantized_dtype`. This parameter cannot be used together with `layer_quant_config`. Example:

```
# auto_hybrid_rate=0.2 means about 20% of Linear layers are set to w8a16
model = QuantTool.quantize(quantized_dtype="w4a16",
                           quantized_method="group32",
                           quantized_algorithm="grq",
                           dataset='./dataset.json',
                           auto_hybrid_rate=0.2,
                           awq_config=None)
```

Export per-layer quantization dtype (export_op_quantized_dtype)

Regardless of which mixed-precision mode is used above, after quantization is complete, you need to call `export_op_quantized_dtype` to export the actual quantization dtype used by each layer to a JSON file for alignment in the subsequent `rknn.build` stage. This API requires the path of the exported ONNX model to map PyTorch layer names to ONNX operators; the exported JSON is in `{onnx_op_name: dtype}` format, where the key is the name of an operator such as MatMul/Gemm in the ONNX model, and the value is the quantization dtype of that operator. Taking Qwen3.5 as an example, the exported `layer_bit.json` content is as follows:

```
{
  "onnx::MatMul_8968": "w8a16",
  "onnx::MatMul_8973": "w8a16",
  "onnx::MatMul_9657": "w8a16",
  ...
}
```

Example:

```
# Call after exporting ONNX; pass in the ONNX model path
QuantTool.export_op_quantized_dtype(onnx_path='./Qwen3.5-0.8B-llm.onnx',
                                    op_dtype_path='layer_bit.json')
```

Corresponding changes when exporting an RKNN model

Since mixed-precision quantization has already been completed in the RKQuantizer stage (the ONNX weights have been fake-quantized according to each layer's type), when exporting an RKNN model through `rknn.config` + `rknn.load_llm` + `rknn.build`, you need to pass `layer_bit.json` through the `op_quantized_dtype` parameter of `rknn.config` so that the RKNN conversion stage restores each layer according to its specified quantization dtype, avoiding accuracy regression caused by re-quantizing sensitive layers with a lower bit width during conversion. `op_quantized_dtype` accepts a dictionary or a JSON file path, currently only takes effect for MatMul/Gemm/Linear operators, and the supported values are w6a16 or w8a16.

```
rknn.config(
    target_platform='rk1820',
    quantized_dtype='w4a16',
    quantized_algorithm='normal',
    quantized_method='group32',
    llm_config=llm_config,
    op_quantized_dtype='./layer_bit.json',   # Pass in the per-layer quantization
dtype from mixed precision
)
```

Notes:

1. `layer_quant_config` and `auto_hybrid_rate` cannot be used together.
2. `export_op_quantized_dtype` must be called after the ONNX export is complete, and the passed `onnx_path` should be the same ONNX model later loaded by `rknn.load_llm`; otherwise the mapping from PyTorch layer names to ONNX operators will fail.
3. When exporting RKNN, the JSON specified by `op_quantized_dtype` must correspond to the file exported by `export_op_quantized_dtype` during the quantization stage. A missing one of the two will cause the mixed-precision configuration to not take effect or cause a conversion error.
4. When mixed-precision quantization is not enabled, there is no need to call `export_op_quantized_dtype`, nor to configure `op_quantized_dtype` in `rknn.config`.

11.5 GRQ Quantization Memory Usage

The GRQ quantization algorithm fine-tunes the model weights. Its execution involves the forward inference and backpropagation of a single transformer block, so the GPU memory usage mainly comes from the forward/backward computation of a single block, rather than the entire model residing in memory at once.

Key factors affecting memory

- **Number of calibration samples**: GRQ fine-tunes block by block and sample by sample. The peak memory is determined by a single computation of a single block, and is basically independent of the number of samples. Therefore, increasing the number of samples does not significantly raise the peak memory, and the number does not need to be large — **generally no more than 200 samples are needed**.
- **Number of tokens in calibration samples**: The token length of a sample is positively correlated with memory usage. Since the attention computation of a Transformer has $O(T^2)$ complexity, a larger token count leads to faster growth of intermediate activations and memory in a single forward/backward pass, which is the main factor determining peak memory. Therefore, when preparing calibration data, you should reasonably control the token length of each sample according to the actual memory capacity of the GPU.

Methods to reduce memory usage

Specifying FlashAttention2 when loading the model can effectively reduce the memory usage during the GRQ stage (FlashAttention reduces the $O(T^2)$ attention memory overhead to $O(T)$). It is recommended to enable it when memory is limited. To enable it, set `_attn_implementation="flash_attention_2"` in `from_pretrained`, for example:

```
model = AutoModelForCausalLM.from_pretrained(
    args.model_path,
    torch_dtype=torch.bfloat16,  # Enabling FlashAttention2 requires the model to
be loaded as bfloat16
    _attn_implementation="flash_attention_2",   # Enable FlashAttention2 to reduce
GRQ memory usage
    trust_remote_code=True
)
```

Note: FlashAttention2 is only used in the GRQ quantization stage to reduce memory. Before exporting to ONNX, the attention implementation must be reset to eager mode (ONNX export does not support FlashAttention), for example:

```
# Switch attention back to eager before exporting ONNX
model.config._attn_implementation = "eager"
for layer in model.model.layers:
    layer.self_attn.config._attn_implementation = "eager"
```

Measured memory reference

The following is the measured memory usage under different attention implementations (for reference). When FlashAttention2 is not enabled (default eager attention), the $O(T^2)$ memory

overhead of attention computation cannot be reduced to O(T), and the memory usage will be significantly higher than when it is enabled. For GPUs with limited memory, it is recommended to enable FlashAttention2:

Model	Calibration sample token length	FlashAttention2 enabled	Without FlashAttention2 (default eager)
Qwen3-4B	~13K	~7G	~20G
Qwen3.5-4B	~13K	~15G	~30G